# WHO KNOWS...
# THE GAME IS THE GAME

A REPORT SUBMITTED TO THE UNIVERSITY OF MANCHESTER
FOR THE DEGREE OF BACHELOR OF SCIENCE
IN THE FACULTY OF SCIENCE AND ENGINEERING

2026

Student id: 11372282

Department of Computer Science

# Contents

**Abstract** **8**

**Declaration** **9**

**Copyright** **10**

**Acknowledgements** **11**

**1 Introduction** **15**
1.1 Overview . . . . . 15
1.2 Motivation . . . . . 16
1.3 Project Aims and Objectives . . . . . 16
1.4 Success Criteria . . . . . 17
1.5 Evaluation Approach . . . . . 18
1.6 Report Structure . . . . . 18

**2 Background** **20**
2.1 Artificial Neural Networks . . . . . 20
2.1.1 What is an Artificial Neural Network? . . . . . 20
2.1.2 Training an Artificial Neural Network . . . . . 20
2.2 Convolutional Neural Networks for Object Detection . . . . . 21
2.2.1 Convolutional Feature Extraction . . . . . 21
2.2.2 Residual Learning . . . . . 21
2.2.3 Object Detection & Bounding Box Regression . . . . . 21
2.2.4 GIoU Loss . . . . . 22
2.3 Spiking Neural Networks . . . . . 22
2.3.1 The Leaky Integrate-and-Fire Neuron . . . . . 23
2.3.2 Surrogate Gradients . . . . . 23
2.3.3 Temporal Dynamics & Rapid Categorisation . . . . . 24
2.4 Event-Based Vision . . . . . 24
2.4.1 Event Cameras . . . . . 24
2.4.2 The Natural SNN–Event Camera Pairing . . . . . 25

2.5 ETraM Dataset . . . 26
2.5.1 Dataset Overview & Acquisition Setup . . . 26
2.5.2 Class Taxonomy, Scale & Annotation . . . 26
2.5.3 Suitability for This Project . . . 26
2.6 Neuromorphic Hardware & Energy Modelling . . . 27
2.6.1 Neuromorphic Hardware Platforms . . . 27
2.6.2 The MAC vs AC Energy Distinction . . . 27
2.7 Related Work . . . 27
2.7.1 The Progression of Spiking Architectures . . . 27
2.7.2 Event-Based Object Detection . . . 28
2.7.3 The Gap This Project Addresses . . . 28

**3 Technical Quality, Methodology & Evaluation 29**
3.1 Requirements Analysis . . . 29
3.1.1 Functional Requirements . . . 29
3.1.2 Non-Functional Requirements & Constraints . . . 30
3.2 System Architecture Overview . . . 31
3.3 Data Preprocessing: The Box-aligned Offline Label-aware Tensorisation (BOLT) Pipeline . . . 33
3.3.1 Raw Event Ingestion & Windowing . . . 34
3.3.2 Signal Conditioning . . . 35
3.3.3 Spatiotemporal Voxelization . . . 35
3.3.4 Downsampling & Normalization . . . 35
3.3.5 Multi-Object Label Alignment . . . 36
3.3.6 Chunking for High-Throughput I/O . . . 38
3.3.7 BOLT Pipeline: Visualized . . . 38
3.4 Spiking Neural Network (SNN) Architecture . . . 40
3.4.1 Spatiotemporal Input & Multi-Step Execution . . . 40
3.4.2 Neuron Dynamics: Leaky Integrate-and-Fire (LIF) . . . 40
3.4.3 Backbone: SEW-ResNet & Attention . . . 41
3.4.4 Firing Rate Regularization (FRR) . . . 43
3.4.5 Multi-Box Regression Head . . . 44
3.5 Convolutional Neural Network (CNN) Baseline . . . 47
3.5.1 Structural Configuration & Layer-Wise Mapping . . . 47
3.5.2 Input Strategy: Channel-Wise Temporal Integration . . . 47
3.5.3 Backbone: Identity Mapping & Residual Connections . . . 48
3.5.4 SEWResBlock vs Standard ResBlock . . . 48
3.5.5 Attention Mechanism: SE-Block . . . 49
3.5.6 The “Gold Standard” Upper Bound . . . 49

3.6 Architectural Evolution . . . 51
3.6.1 Phase 0: The Spatial Cropping Pitfall (Early Prototyping) . . . 51
3.6.2 Generation 1 (G1): The Multi-Box Paradigm Shift . . . 52
3.6.3 Generation 2 (G2): The Expanded Spatial Head (7.00M Parameters) . . . 54
3.6.4 Generation 3 (G3): The High-Resolution Backbone (11.7M Parameters) 55
3.6.5 Generations: Summarized . . . 56
3.7 Training Structure . . . 57
3.7.1 Objective Function . . . 57
3.7.2 Optimisation . . . 57
3.7.3 Validation and Checkpointing . . . 58
3.7.4 Hyperparameter Summary . . . 58
3.8 Experimental Setup & Benchmarking . . . 60
3.8.1 Dynamic Power Estimation . . . 60
3.8.2 Quantization Error Analysis (Accuracy vs. Object Size) . . . 62
3.8.3 Confidence Calibration . . . 63
3.8.4 Temporal Evidence Accumulation . . . 64
3.8.5 Real-Time Latency Benchmark . . . 65
3.8.6 Theoretical Energy Consumption . . . 65
3.8.7 Global Performance Evaluator . . . 66
3.9 Results & Evaluation . . . 68
3.9.1 Global Evaluator . . . 69
3.9.2 Dynamic Power . . . 71
3.9.3 Theoretical Energy Consumption . . . 75
3.9.4 Quantization Error . . . 77
3.9.5 Confidence Calibration . . . 79
3.9.6 Temporal Evidence Accumulation . . . 81
3.9.7 Real-Time Latency Benchmark . . . 83

**4 Summary & Conclusions** **85**
4.1 Summary of Outcomes . . . 85
4.1.1 Global Precision Parity . . . 85
4.1.2 Decisive Efficiency Gains . . . 85
4.1.3 Temporal Reliability and Sharpening . . . 86
4.1.4 Real-Time Viability . . . 86
4.1.5 Success Criteria Validation . . . 86
4.2 Critical Reflection & Future Work . . . 86
4.2.1 Hardware-Paradigm Inequity: The LIF Simulation Penalty . . . 87
4.2.2 The Temporal Quantization Ceiling . . . 87
4.2.3 The Detection Head Mismatch . . . 88

4.2.4 Dataset Domain Gap and Strategic Selection . . . . . 89

**A Implementation Specifications** **94**

A.1 DetectorNX-G3-SNN Hyperparameters . . . . . 95

A.2 DetectorNX-G3-CNN Hyperparameters . . . . . 96

**Word Count: 18373**

# List of Tables


3.1 Success criteria traceability matrix. . . . . 33
3.2 Network Architecture Stages . . . . . 41
3.3 1:1 Architectural Mapping (SNN vs. CNN) . . . . . 47
3.4 Identity Mapping Comparison . . . . . 48
3.5 Generational evolution of the `DetectorNX` architecture. . . . . 56
3.6 Training configuration for `DetectorNX-G3-SNN`. . . . . 59
3.7 Energy constants per operation ($E_{chip}$) for each architecture and platform. . . . 66
3.8 Global Performance Benchmark results for `DetectorNX-G3-SNN` and `DetectorNX-G3-CNN`, evaluated across the full 200-sample ETraM validation set [40]. Ground truth average detections per frame is 2.02 for both models by definition. . . . . 69
3.9 Layer-Wise Spike Activation Rates . . . . . 72
3.10 Energy Statistics per Frame . . . . . 72
3.11 Layer-Wise Operations Comparison . . . . . 75
3.12 Neuromorphic Hardware Energy Extrapolation . . . . . 75
3.13 Comparative efficiency landscape of related SNN detectors. . . . . 77
3.14 Quantization Error Analysis showing drop in Spiking Neural Network (SNN) performance against Convolutional Neural Network (CNN). . . . . 78
3.15 Inference Latency Benchmark . . . . . 83

4.1 Success criteria validation outcomes. . . . . 87

A.1 `DetectorNX-G3-SNN` hyperparameters. . . . . 95
A.2 `DetectorNX-G3-CNN` hyperparameters. . . . . 96

# List of Figures


3.1 High-level system architecture pipeline. . . . . 32
3.2 Overview of the spatiotemporal preprocessing pipeline applied to the ETraM dataset. Raw events are voxelized into a 4D tensor, spatially downsampled, and aligned with multi-object ground truth labels to produce a final sample of shape `[10, 2, 180, 320]` with a corresponding label tensor of shape `[5, 5]`. . . . 39
3.3 The Spike-Element-Wise (SEW) residual block (`SEWResBlock`) as proposed by Fang et al. [10]. . . . . 42
3.4 The Squeeze-and-Excitation (SE) block [17] as applied in Stage 3 of the `DetectorNX-G3-SNN` backbone (22×40×256). . . . . 44
3.5 End-to-end `DetectorNX-G3-SNN` architecture. . . . . 46
3.6 End-to-end `DetectorNX-G3-CNN` architecture. . . . . 50
3.7 Constant Target Bias failure mode . . . . . 52
3.8 Qualitative Inference Audit . . . . . 70
3.9 Dynamic Power Consumption vs. Scene Activity . . . . . 74
3.10 Quantization Error Analysis (Accuracy vs. Object Size) of SNN (Blue) and CNN (Orange). . . . . 78
3.11 Confidence Calibration Reliability Diagrams . . . . . 79
3.12 Temporal Evidence Accumulation Sequences . . . . . 82

# Abstract

Legislation is pushing automotive manufacturers to adopt increasingly complex autonomous driving systems, on a trajectory that will ultimately require full, real-time spatial understanding of the surrounding environment. As a result, detection pipelines have grown substantially in complexity and compute demand, while vehicles remain subject to tight on-board power and thermal budgets. These trends suggest that scaling conventional dense DNNs on GPU-class accelerators is increasingly unsustainable, motivating ultra-efficient architectures such as spiking neural networks for tasks in resource-constrained environments.

We investigate the viability of spiking networks for automotive-grade perception, and measure the cost of the spiking paradigm against a continuous equivalent. We propose DetectorNX: a directly-trained spiking detector (DetectorNX-G3-SNN) designed around activation sparsity to minimise energy consumption while preserving detection accuracy, paired with a structurally equivalent continuous baseline (DetectorNX-G3-CNN) at identical parameter count, trained on the ETraM event-camera dataset to isolate the spiking mechanism as the sole variable. To the best of our knowledge, this is the first directly-trained SNN developed and evaluated on ETraM.

DetectorNX-G3-SNN achieves 94.9% mIoU parity and 91.3% $mAP_{50}$ parity against the continuous baseline, with 100% recall at IoU $> 0.5$. Per-frame theoretical energy consumption drops by 94.33%, scaling linearly with scene activity ($R^2 = 0.7514$). The SNN maintains 259.80 FPS on GPU hardware (8.6$\times$ the automotive real-time threshold) with superior confidence calibration at low-confidence regions.

This project establishes directly-trained SNNs as a viable route to energy-efficient automotive-grade vehicle detection, at a cost of only $\sim 5\%$ Mean Intersection Over Union (mIoU) precision under identical-model SNN–CNN parity. The 1-bit quantisation penalty underlying this gap acts as a scale-independent global ceiling, invalidating the initial scale-dependent hypothesis. Superior low-confidence calibration further positions the SNN as an implicit uncertainty sensor, safety-relevant for autonomous driving. DetectorNX thereby serves as an ETraM baseline, and proposes areas for future work to close the gap with published broader-class spiking detectors.

# Declaration

No portion of the work referred to in this report has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

# Copyright

# Acknowledgements

Firstly, I would like to give thanks to my supervisor, Dr Oliver Rhodes, for his guidance and critical feedback throughout the duration of this project, but also for supporting motivating me to reach the conclusion of this project and degree to the best of my ability. His advice has proved crucial to aspects of this project and report.

Secondly, I want to express my gratitude to my family — namely my parents, Jane and Kanthan, for their endless support and ultimately leading me to this position.

Finally, I would like to thank my friends: Denitta, Gajen, Aden, Neil, Felix, Toby, Tom, and Prav for their support and advice throughout this degree — it has been invaluable.

# Todo list


Check & Proof-read this section against rubric. . . . . 30
Insert images of the super box. . . . . 53
Change last sentence. . . . . 60
Mention SpikeRateMonitor in Evaluation Suite Section and Diagram . . . . . 61
Edit CMOS reference - might not be correct. . . . . 76
Link to future work section. . . . . 81

# List of Abbreviations

**AC** Conditional Addition. 1, 16, 27, 62, 66, 72

**ADAS** Advanced Driver Assistance Systems. 1, 15, 18, 26, 28, 88

**ADP** Average Detections Per Frame. 1, 67

**AMP** Automatic Mixed Precision. 1, 33, 56

**ANN** Artificial Neural Network. 1, 20–23, 27

**ATan** ArcTangent. 1, 24, 41, 57

**BAF** Background Activity Filtering. 1, 35

**BOLT** Box-aligned Offline Label-aware Tensorisation. 1, 3, 26, 30–35, 37–39, 51, 52, 54, 57, 68, 89

**BPTT** Backpropgation Through Time. 1, 24

**CMOS** Complementary Metal-Oxide-Semiconductor. 1, 76

**CNN** Convolutional Neural Network. 1, 6–8, 15–18, 21, 25–28, 33, 47–49, 60–69, 71–73, 75, 76, 78–80, 83–88

**CPU** Central Processing Unit. 1, 65, 68

**CUDA** Compute Unified Device Architecture. 1, 40, 83

**ECE** Expected Calibration Error. 1, 18

**ETraM** Event-based Traffic Monitoring Dataset. 1, 15–18, 26, 28, 33–35, 37, 40, 49, 51, 52, 54, 66, 68, 77, 84, 89

**FLOP** Floating-Point Operation. 1, 15, 61, 65, 66, 75, 76

**FPS** Frames Per Second. 1, 18, 33, 83, 84, 86, 87

**FRR** Firing Rate Regularization. 1, 33, 34, 55, 57, 77

**GIoU** Generalised Intersection over Union. 1, 22, 57

**GPU** Graphics Processing Unit. 1, 17, 18, 27, 38, 65, 66, 68, 76, 84, 86, 87

**HD** High Definition. 1, 16, 28, 34, 35, 89

**HPC** High Performance Computing. 1, 36, 68

**IoU** Intersection over Union. 1, 18, 22, 30, 33, 37, 49, 52–58, 60, 62, 67, 71, 78, 80, 85, 87

**LIF** Leaky Integrate-and-Fire. 1, 3, 4, 18, 23, 25, 34–36, 40, 41, 50, 55, 57, 61, 64, 65, 81, 84, 86, 87

**LRU** Least Recently Used. 1, 38

**MAC** Multiply-Accumulate. 1, 15, 16, 27, 60–62, 66, 72

**$\mathbf{mAP_{50}}$** mean Average Precision at 0.5 IoU threshold. 1, 17, 18, 30, 60, 67, 69, 71, 77, 85, 87, 89

**mIoU** Mean Intersection Over Union. 1, 8, 17, 33, 62, 63, 67, 78, 85, 87, 89

**ReLU** Rectified Linear Unit. 1, 20, 23, 48, 50

**ResNet** Residual Neural Network. 1, 42

**ROC** Rank Order Coding. 1, 88

**SE** Squeeze-and-Excitation. 1, 43, 49, 71, 85

**SEW** Spike-Element-Wise. 1, 7, 27, 42, 71, 85

**SNN** Spiking Neural Network. 1, 6–8, 15–18, 22–28, 33, 34, 36, 40–44, 47–54, 56, 60–69, 71–73, 75–81, 83–89

**SNR** Signal-to-Noise Ratio. 1, 51

**SOP** Synaptic Operation. 1, 17, 27, 35, 56, 61, 65, 66, 72, 75–77

**TTFS** Time-to-First-Spike. 1, 88

**WCET** Worst-Case Execution Time. 1, 83, 84

# Chapter 1

# Introduction

## 1.1 Overview

Autonomous perception is one of the most critical frontiers in automotive engineering, driven by consumer demand and regulatory mandates for Advanced Driver Assistance Systems (ADAS). However, the pursuit of full autonomy has driven a relentless escalation in the complexity and resolution of these perception systems: modern vehicles rely on Convolutional Neural Networks (CNNs) to process high-definition video streams in real-time, executing billions of dense, Floating-Point Operations (FLOPs) every second.

As the industry scales towards higher resolutions and faster reaction times to ensure passenger safety, the energy demands of this continuous, dense spatial reasoning have grown substantially. Lin et al. [23] documented CPU-based CNN inference exceeding the thermal and power constraints of Advanced Driver Assistance Systems (ADAS) hardware, motivating the industry's shift to Graphics Processing Unit (GPU)-class accelerators; continued growth in perception complexity now places dense Artificial Neural Network (ANN) workloads on GPUs on a similar trajectory, motivating a paradigm shift towards fundamentally more efficient architectures.

This project investigates utilizing cutting-edge neuromorphic paradigms, more specifically Spiking Neural Networks (SNNs), and their feasibility in this domain as an alternative computational approach. To achieve this, the project develops `DetectorNX-G3-SNN`, a high-definition spiking architecture designed for multi-object automotive perception. By rigorously benchmarking this system against a strictly equivalent continuous CNN on the event-based Event-based Traffic Monitoring Dataset (ETraM) dataset [40], this research empirically deconstructs the performance-efficiency frontier—measuring the precise trade-offs between the energy savings of event-driven dynamics and the spatial precision constraints of 1-bit temporal quantization.

## 1.2 Motivation

Beyond the generic scaling pressure noted in Section 1.1, Convolutional Neural Networks (CNNs) suffer a paradigm-specific weakness. Their static, frame-based nature mandates billions of dense Multiply-Accumulate (MAC) operations per frame regardless of scene content, generating metabolic load and thermal throttling even on visually redundant or static scenes [23].

To resolve these computational and thermal constraints, this research looks toward the most power-efficient perception engine known: the biological brain. Unlike traditional cameras that capture absolute frames at a fixed frequency, biological retinas and neuromorphic event sensors only transmit asynchronous changes in luminance. Consequently, biological neural networks consume metabolic energy only when processing new, relevant information [27].

Spiking Neural Networks (SNNs) are the computational embodiment of this biological sparsity. By replacing continuous, energy-heavy MAC operations with discrete, event-driven Conditional Addition (AC) additions, SNNs offer a theoretical pathway to sub-milliwatt, real-time object detection at the edge. Since spiking neurons remain dormant until triggered by an input event and respond to a stimulus, they naturally bypass the thermal throttling and battery drain issues that plague continuous CNNs [23].

However, a significant gap exists in the current literature regarding the deployment of SNNs for high-definition automotive perception: the transition from 32-bit floating-point CNN activations to 1-bit discrete spikes introduces quantization errors and temporal precision limits. Historically, this has restricted SNNs to low-resolution classification tasks, as they struggle to resolve the fine-grained spatial coordinates required for multi-object bounding box regression in complex traffic scenes.

The `DetectorNX` project is motivated by the critical need to both rigorously quantify and mitigate these trade-offs on modern, high-definition event datasets. By establishing strict architectural parity between an SNN and a CNN baseline, this project seeks to definitively prove whether the immense thermal and power savings of the neuromorphic paradigm can be achieved without fatally compromising the spatial precision required for autonomous vehicle safety.

## 1.3 Project Aims and Objectives

This project pursues a single, well-defined research question: can a SNN, constrained to the same architectural capacity as a continuous CNN, achieve competitive object detection accuracy on high-definition event-based data while delivering meaningful energy efficiency gains? To answer this, the following objectives were established:

1. **Develop a high-definition SNN for automotive multi-object detection:** Design and train `DetectorNX-G3-SNN`, a spiking architecture capable of performing bounding box regression on the High Definition (HD) Event-based Traffic Monitoring Dataset (ETraM) event-camera dataset.

2. **Develop a structurally equivalent CNN baseline:** Construct `DetectorNX-G3-CNN`, a continuous-valued counterpart that mirrors the SNN architecture layer-for-layer with an identical parameter budget, isolating the spiking mechanism as the sole independent variable in all subsequent experiments and evaluations.

3. **Train and evaluate both models under identical conditions:** Apply the same pre-processed dataset, loss function, optimiser, and training duration to both models, ensuring that any observed performance differences are attributable solely to the computational paradigm.

4. **Benchmark inference latency on GPU hardware:** Measure the real-time throughput of both architectures to assess their viability for deployment in automotive perception pipelines.

5. **Estimate theoretical energy consumption:** Derive energy usage from layer-wise spike activity using established neuromorphic hardware constants, and project consumption figures onto representative neuromorphic platforms such as Intel Loihi and IBM TrueNorth.

6. **Conduct a rigorous, multi-dimensional comparison:** Synthesise the accuracy, latency, and energy results to empirically characterise the performance-efficiency frontier of the spiking paradigm on a real-world, high-definition detection task.

## 1.4 Success Criteria

To provide a concrete evaluation framework, the following measurable success criteria were defined prior to experimentation. Each criterion is revisited explicitly in Chapter 4.

1. **Detection accuracy parity:** On both mIoU and mean Average Precision at 0.5 IoU threshold ($mAP_{50}$), the SNN must score at least 90% of the CNN baseline on the ETraM validation set. Achieving this will validate that our architecture succeeds in mitigating the compromises expected by the 1-bit temporal quantisation error, across both the geometric-precision metric (mIoU) and the stricter COCO-convention detection-quality metric ($mAP_{50}$).

2. **Measurable theoretical energy savings:** The SNN must demonstrate a reduction in theoretical energy consumption per inference relative to the CNN baseline, computed from layer-wise Synaptic Operation (SOP) counts and published neuromorphic hardware constants. The reduction in energy consumption must be realised on real GPU hardware, even if it is not to the same extent as the extrapolated values on neuromorphic hardware.

3. **Real-time viability:** Both models must sustain an inference throughput of at least 30 frames per second on GPU hardware, meeting the established minimum threshold for real-time automotive perception [20].

4. **Architectural parity:** The SNN and CNN must share an equivalent parameter count (within 1%), ensuring that any observed performance gap reflects the spiking mechanism rather than differences in model capacity.

5. **Detection completeness:** Both models must achieve a recall of 100% for objects with an Intersection over Union (IoU) exceeding 0.5 on the validation set, confirming that the multi-box regression head reliably detects all ground-truth vehicles in the scene.

6. **Confidence calibration:** The SNN must exhibit confidence calibration equivalent to, or superior to, that of the CNN baseline on the ETraM validation set. This criterion is safety-critical: in ADAS applications, a model that systematically over-states its detection confidence may suppress downstream safety interventions. A neuromorphic architecture that delivers energy savings at the cost of increased over-confidence would therefore be unsuitable for deployment, regardless of its accuracy or efficiency gains.

## 1.5 Evaluation Approach

Evaluation is designed to isolate the effect of the spiking mechanism as precisely as possible. Because `DetectorNX-G3-SNN` and `DetectorNX-G3-CNN` are parameter-matched and trained under strictly identical conditions, any observed differences can be attributed to the computational paradigm rather than to capacity or training asymmetries. All experiments are conducted on a validation split of the ETraM dataset, built using the stratified sampling strategy described in Section 3.9, to ensure that day, night, rain, and clear-weather scenes are proportionally represented.

Two complementary dimensions are measured. *Prediction Performance* captures how accurately each model localises vehicles in the event stream, using IoU, $mAP_{50}$, recall at a fixed overlap threshold, and Expected Calibration Error (ECE)-based confidence calibration. *Model Performance* captures deployment efficiency, using inference latency on GPU hardware against the 30 Frames Per Second (FPS) real-time threshold [20], and a theoretical energy model projected onto neuromorphic targets. Detailed formulations, hypotheses, and the analytical energy framework are presented with their experiments in Section 3.8.

## 1.6 Report Structure

In the following chapters, we cover the following:

**Background:** Chapter 2 provides the theoretical context for the `DetectorNX` project. It surveys CNNs for object detection, introduces SNNs via the LIF neuron and surrogate gradients, and reviews event-based vision alongside the ETraM dataset. It then characterises neuromorphic

energy economics, concluding with a collection of related work that identifies the critical gap in automotive-specific spiking object detection at high-definition.

**Methodology:** Chapter 3 presents the full technical implementation. It details the system requirements, the shared multi-box detection head architecture, the data preprocessing pipeline, the design of `DetectorNX-G3-SNN` and `DetectorNX-G3-CNN`, the architectural evolution that led to the final configurations, and the complete experimental setup including training hyperparameters, hardware environment, and evaluation protocol. Results and their analysis are presented within this chapter as an integrated discussion of methodology and outcome.

**Conclusions:** Chapter 4 concludes the report. It revisits each success criterion defined in Section 1.4, summarises the principal findings across the accuracy, latency, and energy dimensions, reflects critically on the limitations of the experimental design, and proposes concrete directions for future work including deployment on physical neuromorphic hardware and exploration of alternative spiking encodings.

# Chapter 2

# Background

## 2.1 Artificial Neural Networks

### 2.1.1 What is an Artificial Neural Network?

ANNs are a family of computational models loosely inspired by the structure of biological neural tissue [22]. An ANN is composed of layers of interconnected nodes, or neurons, each of which computes a weighted sum of its inputs and passes the result through a non-linear activation function. By stacking multiple of these layers in sequence, the network learns a hierarchical composition of transformations that progressively maps raw input data to a task-specific output representation. The dominant activation function in modern deep learning is the Rectified Linear Unit (ReLU), defined as $f(x) = \max(0, x)$, which is computationally inexpensive and empirically effective at enabling deep networks to learn complex, non-linear functions [22].

### 2.1.2 Training an Artificial Neural Network

Training an ANN is performed by minimising a differentiable loss function over the training set via gradient descent. Gradients are propagated from the output layer back through the network using the chain rule of calculus, using a procedure known as backpropagation [22]. A fundamental requirement of this process is that every function in the computational graph must be differentiable: if a non-differentiable operation is encountered, the chain rule breaks down and no gradient signal can propagate through it. This constraint is unproblematic for standard ANNs where activations such as ReLU are differentiable almost everywhere, but becomes a significant challenge for the spiking paradigm, which is non-differentiable, as discussed in Section 2.3.

## 2.2 Convolutional Neural Networks for Object Detection

### 2.2.1 Convolutional Feature Extraction

A Convolutional Neural Network (CNN) extends the standard ANN architecture by replacing dense linear layers with convolutional operations, in which a bank of learned filters is applied across the spatial extent of an input. Each filter performs a localised weighted sum at every spatial position, producing a feature map that encodes the presence and magnitude of a particular learned pattern at each location. Stacking these layers yields a representational hierarchy, with early layers capturing low-level edges and motion boundaries and deeper layers composing them into object-level features [22].

### 2.2.2 Residual Learning

A well-documented failure mode of deep CNNs is the *vanishing gradient* problem: as back-propagation proceeds through many successive layers, gradients are repeatedly scaled by values less than one, causing them to decay exponentially and preventing effective learning in early layers [15]. He et al. [15] resolved this through the introduction of *residual learning*: rather than tasking each layer with learning a direct mapping $y = \mathcal{H}(x)$, the network instead learns the *residual* $\mathcal{F}(x) = \mathcal{H}(x) - x$, recasting the target as:

$$y = \mathcal{F}(x) + x \tag{2.1}$$

The additive shortcut connection, otherwise known as a skip connection, bypasses one or more convolutional layers and adds the input directly to the output, providing an unobstructed gradient path back to the earliest layers, guaranteeing stable training regardless of network depth. Within a Standard Residual Block, this shortcut carries the raw input features forward through the identity path, and the convolutional branch learns only the incremental refinement needed to improve the representation. This mechanism forms the backbone of the CNN baseline employed in this project. In a spiking network, however, the standard residual formulation fails, since naively summing two binary spikes produces a non-binary value; a variant that preserves binary output therefore underpins the `DetectorNX-G3-SNN` architecture, as detailed in Section 3.4.3.

### 2.2.3 Object Detection & Bounding Box Regression

Object detection requires a network to simultaneously localise and identify one or more targets within a scene, and determine what its coordinates and size in the space are. In the bounding box regression paradigm popularised by Redmon et al. [28], this is framed as a direct regression problem: the network predicts a set of candidate boxes, each parameterised by a feature tuple $\mathcal{F} = \{x, y, w, h, c\}$, where $(x, y)$ and $(w, h)$ are the normalised centre coordinates and dimensions

of the box, and $c$ is an objectness confidence score reflecting the probability that a genuine target is present within that region.

By predicting multiple candidate boxes simultaneously in a single forward pass, this multi-box formulation enables real-time detection across the full spatial extent of the input – a necessity for automotive perception, where multiple vehicles and artifacts may coexist in the same scene. Each predicted box is evaluated against the ground-truth annotations using an Intersection over Union (IoU) metric, which quantifies the ratio of the intersection to the union of the two boxes, providing a scale-invariant, directly interpretable measure of spatial overlap. The multi-box regression paradigm, including the $(x, y, w, h, c)$ output format and the use of IoU as the evaluation criterion, forms the direct foundation of the detection head employed in this project, as detailed in Section 3.4.5.

### 2.2.4 GIoU Loss

Whilst standard IoU is an effective evaluation metric, it presents a fundamental problem when used directly as a training loss: when a predicted box and its ground-truth target do not overlap at all, the IoU score is zero regardless of how far apart the two boxes are. Consequently, the gradient of a standard IoU loss is also zero in this case, providing no optimisation signal to guide the network towards the correct region and stalling training in the early stages when predictions are furthest from the ground truth. Rezatofighi et al. [29] resolved this by proposing Generalised Intersection over Union (GIoU), which augments the standard overlap term with a penalty derived from the area of the smallest enclosing convex hull $\mathcal{C}$ that contains both boxes:

$$\text{GIoU} = \text{IoU} - \frac{|\mathcal{C} \setminus (A \cup B)|}{|\mathcal{C}|} \tag{2.2}$$

This penalty is non-zero even when the boxes are entirely disjoint, ensuring a meaningful gradient signal is always available. As the predicted box converges towards the ground truth, the Generalised Intersection over Union (GIoU) penalty vanishes and the loss reduces to standard IoU, making it a strict generalisation of the original metric. GIoU is employed as the training loss throughout this project, with standard IoU being reserved for evaluation, where interpretability and comparability with published results take precedence over gradient behaviour.

## 2.3 Spiking Neural Networks

SNNs represent a fundamentally different computational paradigm to the standard ANN architectures described in the preceding sections. Rather than propagating continuous-valued activations between layers, SNNs communicate exclusively through discrete binary events, spikes, mimicking the all-or-nothing firing behaviour of biological neurons [30]. By prioritising event-driven binary communication, this paradigm aims to drastically reduce both dynamic

power consumption and computational complexity compared to traditional deep learning models, making it inherently suited to power and latency-constrained environments.

### 2.3.1 The Leaky Integrate-and-Fire Neuron

At the foundation of most practical SNNs lies the Leaky Integrate-and-Fire (LIF) neuron model, which governs the temporal dynamics and binary spike generation of the network. Unlike standard ANNs that utilise continuous activation functions such as ReLU, the LIF node maintains an internal state: the membrane potential $V[t]$, which updates iteratively across discrete timesteps $t \in [1, T]$.

The dynamics of the LIF neuron at a given spatial position and timestep $t$ are governed by the integration of incoming pre-synaptic spatial features $X[t]$ and an inherent leak factor $\tau$, which dictates how quickly $V[t]$ returns to its resting state in absence of incoming input. The hidden membrane potential $H[t]$ prior to spiking is mathematically defined as:

$$H[t] = V[t-1] + \frac{1}{\tau}\left(X[t] - (V[t-1] - V_{reset})\right) \tag{2.3}$$

When $H[t]$ exceeds a predefined threshold $V_{th}$, the neuron emits a binary spike $S[t] \in \{0, 1\}$ to the subsequent layer, governed by the Heaviside step function $\Theta(x)$:

$$S[t] = \Theta(H[t] - V_{th}) \tag{2.4}$$

Following a spike ($S[t] = 1$), the neuron undergoes a hard reset, instantly dropping its potential $V[t]$ to $V_{reset}$, mimicking the refractory period of biological neurons, preventing infinite spiking and overstimulation [31]. If no spike is emitted ($S[t] = 0$), the potential naturally decays, ensuring that subsequent layers only perform computations upon receiving active events [32]. This *activation sparsity*, the property that computation is triggered only by genuine signal, not by silence, is the primary driver of the SNN's exceptional energy efficiency [30].

### 2.3.2 Surrogate Gradients

Training deep SNNs via backpropagation presents a fundamental mathematical challenge, since the Heaviside step function governing spike generation is non-differentiable. As established in Section 2.1, the chain rule requires every function in the computational graph to be differentiable; the Heaviside function violates this directly, since its derivative is zero everywhere except at the threshold $V_{th}$, where it is infinite. Using conventional backpropagation therefore results in zeroed gradients that immediately halt the learning process, a phenomenon known as the "dead neuron" problem.

To resolve this, SNNs employ a *surrogate gradient* during the backward pass [26]. While the forward pass remains strictly binary, the backward pass approximates the step function using a

smooth, differentiable curve. A common choice is the ArcTangent (ATan) function:

$$\frac{\partial S}{\partial H} \approx \frac{\alpha}{2\left(1+\left(\frac{\pi}{2}\alpha(H-V_{th})\right)^2\right)} \tag{2.5}$$

where $\alpha$ is a hyperparameter controlling the steepness of the surrogate curve. This mathematical substitution allows the network to learn complex spatiotemporal representations from scratch while maintaining a purely binary forward execution, with gradients propagated through the temporal dimension via Backpropgation Through Time (BPTT).

### 2.3.3 Temporal Dynamics & Rapid Categorisation

A defining property of SNNs is their natural ability for spatiotemporal integration, combining timing and location when processing information. By processing an event stream iteratively across $T$ discrete timesteps, an SNN is capable of producing meaningful predictions before the full observation window has elapsed [27].

This progressive refinement is analogous in spirit to the *rapid categorisation* phenomenon documented in the human visual system, where early feedforward spikes drive coarse semantic decisions that sharpen as further neural evidence accumulates [37]. While Thorpe, Fize and Marlot [37] address categorical recognition rather than coordinate regression, the underlying principle that sparse early spike activity produces meaningful but imprecise representations which sharpen over time is directly consistent with the anticipated behaviour of a spiking detection backbone.

Rate coding over a fixed window of $T$ timesteps carries an inherent representational limit known as the ”quantisation error”; the precision of the encoded signal is bounded by a $1/T$ quantisation floor, since information is encoded as spike counts that can take only $T+1$ discrete values [36].

## 2.4 Event-Based Vision

### 2.4.1 Event Cameras

#### Frame-Based vs. Event-Based Sensing

Conventional frame-based cameras operate synchronously, where every pixel is read out at a fixed rate, producing a dense intensity matrix regardless of whether anything in the scene has changed since the last frame. This design introduces several well-documented limitations: motion blur at high speeds, a fixed dynamic range bounded by the exposure window, and substantial data redundancy during static scenes — all of which are particularly problematic for high-speed automotive perception [12].

An event camera fundamentally departs from this paradigm; rather than reading out all pixels simultaneously, each pixel operates independently, firing an event the moment its log-intensity changes beyond a threshold. The result is a continuous, asynchronous stream of discrete events rather than a sequence of synchronous frames, registering only what has changed between scenes instead of the full scenes themselves.

#### Advantages and the Compatibility Problem

This per-pixel, threshold-driven design gives event cameras a compelling profile for autonomous perception: high temporal resolution, high dynamic range, low latency, and low data redundancy [12]. However, a sparse, unordered point cloud of $(x, y, t, p)$ tuples is fundamentally incompatible with conventional deep learning frameworks, which expect dense, structured tensors of fixed spatial dimensions. Correcting this to adapt to deep learning approaches requires a preprocessing step to convert the asynchronous stream into a format suitable for neural network ingestion, the specific design of which is detailed in Section 3.3.

### 2.4.2 The Natural SNN–Event Camera Pairing

#### An Event-Driven Architecture for an Event-Driven Sensor

Both systems are fundamentally event-driven: both remain silent in the absence of stimuli and emit discrete binary events only in response to genuine activity.

A CNN must first collapse the event stream into a single dense synchronous tensor before it can be processed, discarding all temporal structure. In contrast, an SNN processes the preprocessed spatiotemporal tensor iteratively across $T$ timesteps through its LIF dynamics, preserving the event-driven dynamics [12, 27]. An SNN can therefore perform early-exit inference, emitting predictions before the full $T$-step window has elapsed, whereas a CNN must wait for its complete aggregated tensor.

#### Implications for Neuromorphic Deployment

On neuromorphic hardware such as Intel Loihi [4] or IBM TrueNorth [25], this alignment extends to the physical level. An event camera connected directly to a neuromorphic chip can route events to spiking neurons without any voxelization or dense tensor conversion, since the raw events can be routed directly to spiking neurons without any intermediate tensor representation, enabling a fully asynchronous end-to-end inference pipeline and eliminating the preprocessing bottleneck entirely.

## 2.5 ETraM Dataset

### 2.5.1 Dataset Overview & Acquisition Setup

ETraM is a first-of-its-kind, fully event-based traffic monitoring dataset introduced by Verma et al. [40] at CVPR 2024. Data was captured using the Prophesee EVK4 HD event camera at $1280 \times 720$ spatial resolution with a temporal resolution exceeding 10,000 fps, mounted statically at approximately 6 m height with a 35° pitch angle. Unlike ego-motion datasets, where the camera itself moves through the scene, this dataset provides a static perspective; a fixed vantage point ensures that events are generated predominantly by moving traffic participants rather than background infrastructure. Recordings span intersections, roadways, and local streets around Arizona State University, Tempe campus, totalling ten hours of annotated event data.

### 2.5.2 Class Taxonomy, Scale & Annotation

ETraM covers eight distinct object classes organised into three broad categories: Vehicles (Car, Truck, Bus, Tram), Pedestrians, and Micro-mobility (Bicycle, Bike, Wheelchair). Over 2M bounding boxes were manually annotated using CVAT [40], with each annotation additionally carrying an object ID to support multi-object tracking. Raw data is distributed in multiple formats (RAW, DAT, and H5), with ground-truth annotations provided in NumPy format. For this project, the dataset was abstracted to the Vehicle category exclusively, allowing the SNN–CNN comparison to be isolated on a single, well-defined object class.

### 2.5.3 Suitability for This Project

Two properties motivated the selection of ETraM for this project. First, critically for an ADAS and autonomous driving context, recordings span the full spectrum of real-world operating conditions: daytime, nighttime, and twilight, across sunny, overcast, and rainy weather - precisely the range of environments an autonomous perception system must handle reliably, and one that deliberately exercises the conditions under which frame-based sensors are known to degrade. Second, the ETraM study provides a dataset-specific spatiotemporal filtering and denoising pipeline tuned to the noise characteristics of the EVK4 sensor, which was adopted directly as the signal conditioning stage of the BOLT preprocessing pipeline. This eliminated the need to develop bespoke noise suppression from first principles, and significantly accelerated the transition to model development. The full BOLT pipeline is detailed in Section 3.3.

## 2.6 Neuromorphic Hardware & Energy Modelling

### 2.6.1 Neuromorphic Hardware Platforms

Intel Loihi [4] and IBM TrueNorth [25] represent the two principal reference platforms for neuromorphic computing. Unlike conventional GPUs, which execute computation synchronously across all neurons at every clock cycle, both architectures operate asynchronously and event-driven. A neuron consumes energy only when it receives a spike and crosses its firing threshold, remaining entirely silent, drawing no dynamic power in the absence of input activity. This mirrors the SNN's spike-based dynamics precisely, making neuromorphic hardware the natural target deployment environment for event-driven architectures and completing the end-to-end alignment with the event camera described in Section 2.4.2. The per-SOP energy costs of these platforms, 23.6 pJ for Intel Loihi [4] and 26.0 pJ for IBM TrueNorth [25], form the hardware extrapolation targets used in the energy evaluation of this project.

### 2.6.2 The MAC vs AC Energy Distinction

The foundational energy analysis underlying this project rests on the silicon-level operation costs documented by Horowitz [16]. A standard CNN performs a full MAC at each synaptic connection - a floating-point multiplication followed by an accumulation, costing $E_{\text{MAC}} = 4.6\,\text{pJ}$ per 32-bit operation. An SNN, by contrast, communicates exclusively via binary spikes; since a spike is either 0 or 1, the multiplication is eliminated entirely, reducing each synaptic event to an Conditional Addition (AC) operation at a cost of $E_{\text{AC}} = 0.9\,\text{pJ}$ - a $5\times$ reduction in per-operation energy at the silicon level [16]. This asymmetry is the first of two complementary mechanisms underpinning the SNN's energy advantage over its CNN counterpart; the second, input-dependent activation sparsity, is introduced and materialised alongside the complete per-frame energy model in Sections 3.8.1 and 3.8.6.

## 2.7 Related Work

### 2.7.1 The Progression of Spiking Architectures

Early efforts to close the accuracy gap between SNNs and CNNs focused on classification tasks on low-resolution benchmarks. Pfeiffer and Pfeil [27] provided foundational evidence that SNNs could exploit temporal sparsity for energy-efficient inference, while Sengupta et al. [32] demonstrated that deep SNNs could approach CNN accuracy on ImageNet via ANN-to-SNN conversion. The architectural frontier advanced further with the SEW ResNet introduced by Fang et al. [10], which resolved the vanishing gradient problem in directly-trained deep SNNs through element-wise spike residuals. Despite this depth progression, a systematic review of 151 studies by Iaboni and Abichandani [18] confirms that SNN evaluation remains predominantly

anchored to low-resolution datasets such as MNIST (28×28) and CIFAR-10 (32×32). More recent efforts have extended SNNs to multi-object detection at higher resolutions [9, 24, 34, 42], but these architectures target general-purpose benchmarks such as COCO; SNN-based multi-object detection in the automotive domain remains a critically underexplored area.

### 2.7.2 Event-Based Object Detection

The landscape of event-based object detection has been comprehensively surveyed by Gallego et al. [12], who established that tensor-based representations, voxel grids and event surfaces, dominate the field, typically consumed by continuous-valued detectors. The introduction of ETraM by Verma et al. [40] reinforced this pattern; despite providing the first HD event-based traffic dataset, all published baselines, RVT, RED, and YOLOv8, are continuous architectures, and no spiking detector has been evaluated against them. Furthermore, no prior work has systematically compared a spiking and a non-spiking detector on a shared HD event dataset under strict architectural parity.

### 2.7.3 The Gap This Project Addresses

Lin et al. [23] established that CPU-based CNN inference had exceeded the thermal and power constraints of early ADAS systems, motivating the industry's shift to GPU-class accelerators. Since 2018, legislative pressure has driven further growth in perception complexity, placing dense ANN inference on GPUs on a similar trajectory to become unsustainable. Despite this, as Pfeiffer and Pfeil [27] and the event-vision survey of Gallego et al. [12] collectively establish, the neuromorphic alternative has not been rigorously validated at the spatial resolution and domain complexity of real-world automotive perception. The `DetectorNX` project directly addresses this gap: by enforcing strict architectural parity between an SNN and a CNN baseline on the HD ETraM dataset, it provides the first controlled empirical quantification of the precision–efficiency trade-off at the spatial resolution demanded by ADAS systems.

# Chapter 3

# Technical Quality, Methodology & Evaluation

## 3.1 Requirements Analysis

The primary objective of this project was the development and rigorous evaluation of a high-performance Spiking Neural Network (SNN) for multi-object detection using high-definition event-camera data (the ETraM dataset). To effectively quantify the accuracy versus energy efficiency tradeoff between bio-plausible spiking architectures and traditional Convolutional Neural Networks (CNNs), the system had to be designed against a stringent set of functional and non-functional requirements. These requirements guided the iterative evolution of the neural architectures and the underlying computational pipeline.

### 3.1.1 Functional Requirements

To ensure a fair empirical comparison and to solve the complexities of multi-object detection on neuromorphic data, the following functional criteria were established:

- **Multi-Object Regression Capabilities:** Unlike simplified classification tasks, the system was required to perform multi-object detection capable of identifying multiple vehicles simultaneously within a single scene. This necessitated a regression head capable of outputting multiple bounding boxes per frame (defined by spatial coordinates and confidence scores) while actively suppressing "ghost detections" and resolving spatial ambiguities.
- **Architectural Parity (The "Gold Standard" Baseline):** To strictly isolate the impact of the computational paradigm (event-driven spiking versus continuous-valued activations), the baseline CNN was required to be a 1:1 non-spiking counterpart to the SNN. Both models were constrained to equivalent structures, depths and parameter counts ($\sim$ 11.7M parameters). Crucially, to eliminate data variance as a confounding factor, both the SNN and the CNN were trained on the exact same pre-processed neuromorphic dataset (the

BOLT pipeline output). This ensured that any observed discrepancies in performance or power consumption were solely attributable to the underlying network architectures, rather than differing input modalities or varied model capacity.

- **High-Definition Spatiotemporal Ingestion:** The system was required to process raw, high-definition ($1280 \times 720$) event streams. Furthermore, the pipeline had to preserve the temporal dynamics of the data by segregating it into discrete time bins (e.g., $T = 10$) to allow the SNN to demonstrate temporal evidence accumulation.

- **Comprehensive Evaluation Metrics:** The evaluation suite was required to output granular metrics beyond top-line Mean IoU and $\text{mAP}_{50}$. This included real-time dynamic power consumption mapping, confidence calibration analysis (reliability diagrams), and quantization error tracking across varying object sizes.

### 3.1.2 Non-Functional Requirements & Constraints

The processing of high-definition event data at scales of up to 150 training epochs imposed severe computational bottlenecks. Consequently, the system had to satisfy critical performance and stability constraints:

- **Computational Scalability & Speed:** The SNN simulation had to be optimized for execution on local accelerator hardware (GPUs). This required the integration of Automatic Mixed Precision (AMP) to utilize Tensor Cores, memory-efficient vectorized loss functions to eliminate sequential batch loops, and the utilization of optimized C++/CUDA kernels (e.g., SpikingJelly's multi-step modes) over native Python loops.

- **Numerical Stability:** Given the reliance on 16-bit precision through AMP and the deep accumulation of membrane potentials in the SNN, the architecture required specific safeguards against gradient explosion. This constrained the design of the detection head, mandating the use of raw logits and numerically stable loss functions (e.g., `BCEWithLogitsLoss`).

- **Hardware Extrapolation:** While constrained to local GPU execution, the benchmark outputs (e.g., dynamic power tracking) were required to be translatable into theoretical energy consumption metrics (in mJ) based on established hardware constants, facilitating projections for specialized neuromorphic chips like Intel Loihi or IBM TrueNorth.

These requirements formed the foundation for the three-generation evolution of the `DetectorNX-G3-SNN` architecture detailed in Section 3.4, and drove the design of the rigorous benchmarking suite evaluated in Section 3.8.

Check & Proof-read this section against rubric.

## 3.2 System Architecture Overview

To satisfy the success criteria established in Section 1.4 and the functional requirements in Section 3.1, the overall system was designed as an end-to-end, highly modular pipeline. Decoupling the preprocessing, neural backbone, and regression stages enabled the isolation and empirical study of individual architectural modifications, ensuring that any performance gap between the spiking and continuous paradigms is attributable to the paradigm swap itself rather than to differences in preprocessing or head design. The high-level data flow is formalized in Figure 3.1.

As illustrated, the architecture is divided into four primary sequential stages, where the central neural backbone acts as an interchangeable module:

1. **Data Ingestion & The BOLT Pipeline (Fixed):** Raw, asynchronous event data at a high spatial resolution of $1280 \times 720$ is ingested from the ETraM dataset. Due to the conventionally sparse and noisy nature of neuromorphic hardware outputs, the data enters the BOLT preprocessing pipeline, which is heavily adapted from the methodology proposed in the original ETraM study [40]. While the core spatiotemporal noise filtering and bilinear temporal interpolation (to $T = 10$ bins) remain faithful to the original study, this project included an additional spatial downsampling stage ($ds = 4 : 1280 \times 720 \rightarrow 320 \times 180$). This modification was strategically implemented to ensure training feasibility on local accelerator hardware and reduce input sparsity, thereby improving the stability of neuronal feature extraction. This stage is detailed thoroughly in Section 3.3.

2. **The Neural Backbone (Interchangeable):** The preprocessed voxel tensor serves as the input to the primary feature extraction engine. This stage houses either the spiking architecture `DetectorNX-G3-SNN` or its parameter-matched continuous counterpart `DetectorNX-G3-CNN`. Both backbones comprise four residual blocks — `SEWResBlock` units in the SNN variant, Standard ResBlocks in the CNN baseline — with Squeeze-and-Excitation attention at the Stage 3 bottleneck, and together perform an aggressive $8\times$ total spatial downsampling. The evolution and formal design of these backbones are detailed in Sections 3.4 and 3.5.

3. **Multi-Box Regression Head (Fixed):** The final abstracted feature maps from either backbone are fed into a unified detection head. To resolve the spatial ambiguities and "ghost detections" observed in earlier architectural iterations, the head utilizes an $8 \times 8$ Adaptive Pooling mechanism to aggregate features from 64 spatial receptive blocks. This abstracted information is then mapped to a final output tensor of shape $[B, 5, 5]$, representing the top 5 predicted bounding boxes per image. Each detection is defined by the feature set $\mathcal{F} = \{x, y, w, h, c\}$, where $(x, y, w, h)$ denote the normalized bounding box coordinates and $c$ represents the object confidence logit. The regression output is evaluated via a fully vectorized Multi-Box Loss function during training.

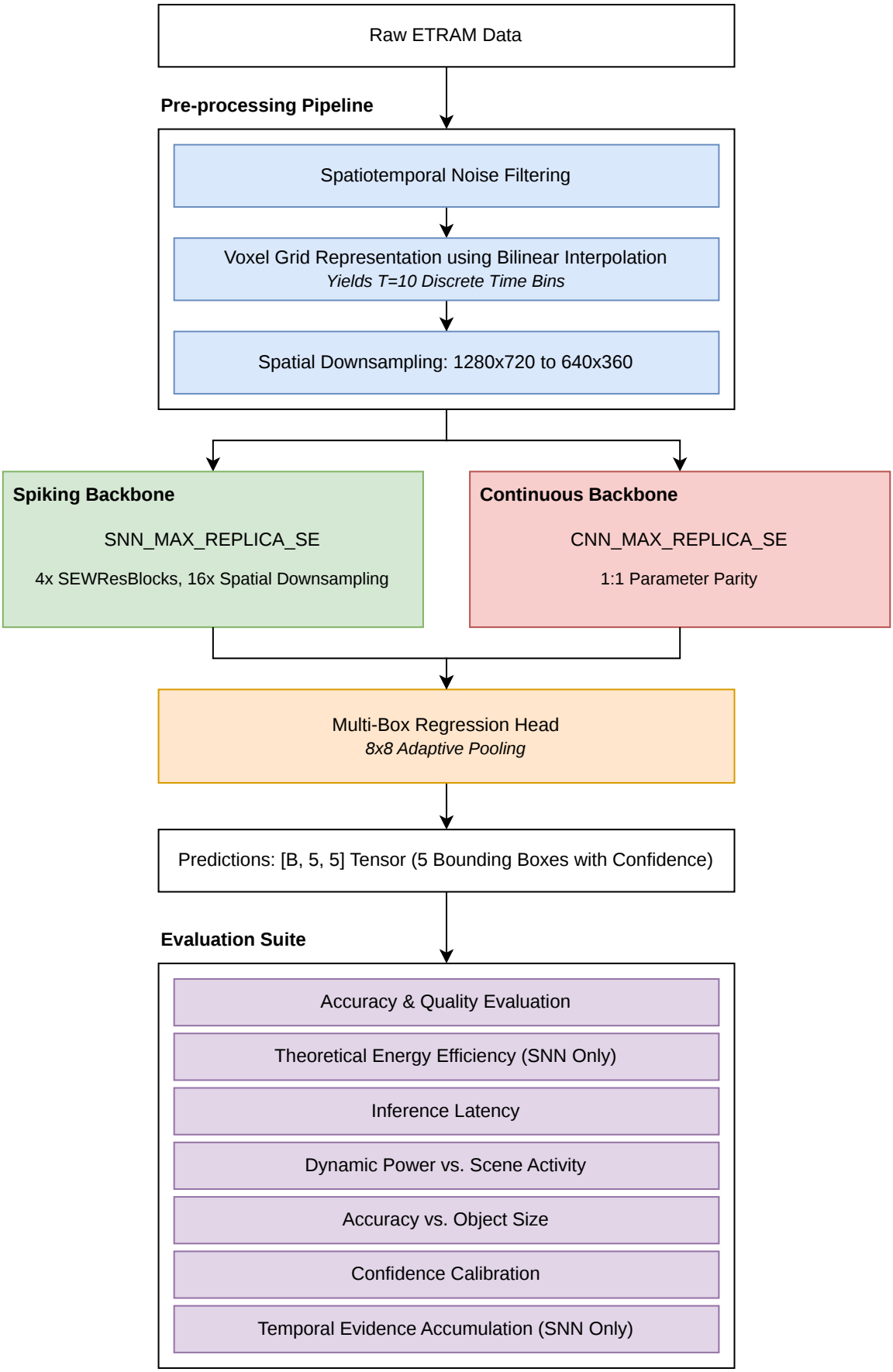



Figure 3.1: The high-level system architecture pipeline, illustrating the flow from raw ETraM high-definition event data, through the fixed BOLT preprocessing stages, into the interchangeable neural backbone (SNN or CNN), and concluding with the multi-box regression head and the comprehensive evaluation suite.

4. **Evaluation & Benchmarking Suite (Fixed):** The predictions and internal neuronal activity from both backbones are passed into a dedicated benchmarking suite that pairs each of the six success criteria with a matched experiment, as summarised in Table 3.1. Diagnostic analyses of quantisation error (Section 3.8.2) [3] and temporal evidence accumulation (Section 3.8.4) complement the primary accuracy result by explaining *how* the SNN's rate-coded regression converges, rather than validating a distinct criterion. The full benchmarking suite is detailed in Section 3.8.

| SC | Success Criterion | Architectural Mechanism | Validating Experiment |
|---|---|---|---|
| SC1 | Detection accuracy parity ($\geq 90\%$ mIoU) | Multi-Box regression head, `SEWResBlock` backbone, SE attention at Stage 3 bottleneck | Global Evaluator (Section 3.8.7) |
| SC2 | Measurable theoretical energy savings | Binary spike encoding, Firing Rate Regularization (FRR) metabolic penalty | Dynamic Power (Section 3.8.1) & Theoretical Energy (Section 3.8.6) |
| SC3 | Real-time viability ($\geq 30$ FPS) | Fused CUDA kernels (`step_mode='m'`), Automatic Mixed Precision (AMP) 16-bit tensor cores | Real-Time Latency (Section 3.8.5) |
| SC4 | Architectural parity (within 1% parameters) | 1:1 block-for-block backbone matching, shared regression head, shared BOLT input | Parameter-count audit (Table 3.6) |
| SC5 | Detection completeness (100% recall @ IoU$>0.5$) | Vectorised Multi-Box Loss with greedy assignment matching | Global Evaluator (Section 3.8.7) |
| SC6 | Confidence calibration equivalence | Objectness logits + `BCEWithLogitsLoss` on unmatched boxes | Confidence Calibration (Section 3.8.3) |

Table 3.1: Mapping of each success criterion (Section 1.4) to the architectural mechanism that addresses it and the dedicated experiment that validates it in Chapter 4.

This Fixed/Interchangeable partition is the mechanism by which SC4 (Architectural Parity) is enforced: all paradigm-level variance between the two evaluated models is confined to Stage 2, so any discrepancy observed in Chapter 4 is attributable to the spike-vs-continuous swap alone.

## 3.3 Data Preprocessing: The BOLT Pipeline

As established in Section 2.4, an event camera outputs a sparse, asynchronous stream of events $e = (x, y, t, p)$, where $(x, y)$ are the spatial coordinates, $t$ is the microsecond timestep, and $p \in \{-1, 1\}$ is the polarity indicating an increase or decrease in brightness. The BOLT preprocessing pipeline takes the raw ETraM data at $1280 \times 720$ spatial resolution and converts it into the dense, structured tensor format required by the SNN and CNN backbones.

### 3.3.1 Raw Event Ingestion & Windowing

The BOLT pipeline begins by reading the raw h5 event files $(x, y, t, p)$ and the partnering npy files containing the bounding box annotations from the ETraM dataset. Once both files have been parsed, the first step taken is to apply a $100ms$ sliding window across the entire duration of the event file. "Windowing" is the process of slicing the continous event stream into chunks that represent a single instance of time for the model, thereby anchoring the asynchronous event stream into discrete, structured training instances.

To maximize training efficiency and ensure a high signal-to-noise ratio in the training distribution, the BOLT pipeline adopts a Box-Central sampling strategy rather than more conventional uniform temporal partitioning, where training windows were anchored deterministically around the centroids of existing ground truth annotations. This approach ensures that 100% of the training voxels contain informative structural targets, mitigating the prevalence of 'empty-scene' noise and providing the SNN with consistent temporal evidence to drive membrane potential accumulation prior to the target timestamp.

In this project, the time window $\Delta T = 100$ms was chosen to align with the temporal persistency of the human visual system of roughly 100ms [6]; just as the human eye integrates luminance changes over approximately one frame of perception, the SNN also integrates the asynchronous event stream over the same duration before producing a detection output.

The practical consequence of this choice is two-fold. First, a 100ms window is long enough to guarantee a minimum event density sufficient for the LIF neurons in the stem layer to accumulate membrane potential and produce meaningful spike trains across all $T = 10$ temporal bins. Second, specific to the context of vehicle detection, it is short enough that a vehicle travelling at highway speed (e.g., 120 km/h $\approx 33$ ms$^{-1}$) displaces by only 3.3m within the window, ensuring that the bounding box annotation assigned to the window remains spatially valid and does not introduce label noise through excessive object displacement.

In combination with the Box-Central sampling approach, the pipeline applies a density filter prior to voxelization: for each 100ms window the raw event count is checked, and windows containing fewer than `min_events` $= 500$ events are discarded. This threshold is a structural necessity for SNN training stability. Near-empty windows collapse the learning signal on two fronts: the forward pass produces few or no spikes, yielding a zero surrogate gradient [26] which causes the optimiser to suppress firing globally [8], while the FRR regulariser, which penalises deviation from a 20% target firing rate [5, 41], dominates the total loss because the target firing rate is unattainable when the input carries no events. A 500-event floor also filters out the baseline Background Activity noise that HD event cameras emit from transistor mismatch and thermal effects [12], ensuring training attention is directed only at windows where meaningful scene change has occurred.

### 3.3.2 Signal Conditioning

The first step of the BOLT pipeline applies a series of signal-conditioning operations, lifted directly from the original ETraM study since they were tuned to the characteristics of the dataset [40], and serving three complementary purposes. Normalising the voxel tensor to $[0,1]$ prevents LIF neuron saturation at the stem layer. Without it, high-magnitude or noisy voxel values drive every neuron to fire continuously and flood the network with uninformative spikes, whereas the constrained range ensures only neurons encoding genuine structural features (vehicle edges, motion boundaries) exceed threshold [8]. The Background Activity Filtering (BAF) denoising step then rejects the low-activity clusters that HD event cameras emit from flickering illumination and sensor noise, freeing representational capacity for real vehicle geometry [8, 30]. Finally, every background event suppressed at preprocessing removes a spike emission at inference, tying preprocessing quality directly to the SOP count on neuromorphic hardware [30].

### 3.3.3 Spatiotemporal Voxelization

To make the asynchronous event data compatible with neural network layers, a Voxelization process is performed to transform the continuous stream into a structured tensor. This process consists of three core operations:

- **Temporal Binning:** The total duration of an event sample (e.g., a 100ms window) is divided into 10 discrete time bins ($T = 10$).
- **Polarity Splitting:** Events are separated into two distinct channels based on their polarity ($C = 2$), representing positive and negative brightness changes.
- **Event Accumulation (Histogramming):** Within each discrete temporal bin and polarity channel, the individual asynchronous events are aggregated at their respective spatial coordinates $(x,y)$. Specifically, the value of each "pixel" in the resulting spatial grid becomes the total count of events (of that specific polarity) that occurred at that exact location during that fraction of time. This process effectively converts the sparse, continuous point-cloud of events into a sequence of dense 2D frames—or event histograms—which standard convolutional and spiking layers can natively process.

This accumulation process preserves the temporal distribution of the events while generating a dense 4D spatiotemporal tensor of shape $[10,2,720,1280]$ (representing Time $\times$ Channels $\times$ Height $\times$ Width).

### 3.3.4 Downsampling & Normalization

Following spatiotemporal voxelization, the voxelized data undergoes a dual-phase refinement consisting of spatial downsampling and amplitude normalization. This is the final polishing step

of the pipeline, transforming the high-resolution event counts into a stabilized, low-memory format that the neural backbones can process efficiently.

#### Spatial Downsampling (16-fold Data Reduction)

The generated high-definition voxel grid ($1280 \times 720$) is subjected to a $4 \times 4$ Max-Pooling operation, reducing the spatial resolution to $320 \times 180$. This 16-fold reduction in data volume is critical for managing the high-dimensional state-space of the SNN backbone. Because SNNs must store neuron membrane potentials across time, an HD model would require massive amounts of GPU memory. By aggressively reducing the spatial dimensions, we ensure the backbone scalability, making it computationally feasible to train the deep 4-block architecture over extended epochs.

Furthermore, utilizing Max-Pooling (as opposed to Average-Pooling) acts as a feature preserver: it retains the strongest event signal within each $4 \times 4$ area, helping the network maintain the crisp, high-contrast edges of the vehicles even at a lower resolution.

It should be noted that the downsampling factor $ds = 4$ was heavily influenced by the High Performance Computing (HPC) cluster's GPU memory constraints; without using $ds = 4$, the data would not fit into the GPU memory, or would require a batch size of 1, making training infeasible within the time constraints of this project.

#### Max-Normalization (Input Calibration)

Subsequently, max-normalization is performed to scale the raw event accumulation counts into a continuous $[0, 1]$ interval, by dividing every pixel in the temporal volume by the single highest event count found within that 10-bin voxel.

This normalization serves as a fundamental safeguard against neural saturation. LIF nodes have fixed firing thresholds (in this architecture, $V_{th} = 0.2$) - if a raw, unscaled event count of 50 were fed into the network, the first-layer neurons would fire instantaneously and repeatedly, effectively "screaming" and drowning out all other spatiotemporal information. By enforcing input calibration, we ensure that the input magnitudes are calibrated relative to the LIF thresholds, facilitating stable surrogate gradient propagation during the 150-epoch training cycle.

### 3.3.5 Multi-Object Label Alignment

The final stage of the pipeline implements a Multi-Object Label Alignment strategy, which is fundamental to the architecture's multi-object detection capabilities. This stage transforms individual, asynchronous bounding box annotations into structured, scale-independent target tensors suitable for the network's multi-box regression head.

### Timestamp-Based Grouping (Unified Scene Representation)

Instead of creating isolated training samples for every individual vehicle, the pipeline parses the ground truth annotations and groups all vehicles that exist at the exact same microsecond ($t$). This process merges individual detections into a Unified Scene Representation, which is crucial because it eliminates "label noise"; presenting the network with identical visual input but different single-car targets across multiple samples would force the model to average its predictions, resulting in blurry, low-confidence bounding boxes with poor IoU scores.

### Fixed-Capacity Padding

Deep learning models inherently require consistent tensor dimensions for batched training. To accommodate scenes with varying numbers of vehicles, we enforce a Fixed-Capacity Interface of exactly 5 bounding boxes per frame. If a scene contains 2 vehicles, the remaining 3 slots are populated with "empty" zero-padded boxes. If a frame contains no vehicles, 5 empty boxes are provided. This ensures every target label is a consistent $[5,5]$ tensor. The limit of 5 boxes was empirically selected based on an analysis of the ETraM validation set, where the vast majority of frames contain 3 or fewer vehicles, making 5 a safe upper bound for this specific traffic monitoring environment.

### Binary Masking

To enable the network to distinguish between genuine vehicle targets and zero-padded instances, the tensor utilizes a Binary Masking dimension. For every box, the target tensor stores five values: $[Center_X, Center_Y, Width, Height, Mask]$.

- For a real object, the mask value is set to 1.0.
- For a padded or empty box, the mask value is set to 0.0.

This mask acts as the objectness confidence target. It is utilized by the Multi-Box Loss function to selectively apply coordinate gradients only to valid targets, while simultaneously regularizing the model's confidence head to output 0.0 for empty regions.

### Normalization to Sensor Space

Finally, all spatial parameters (center coordinates, width, and height) are originally provided in raw pixel values (e.g., $x = 842$). These are normalized by dividing by the raw sensor width (1280) and height (720), scaling all coordinates into the continuous range $[0.0, 1.0]$. This provides a Scale-Independent Signal, ensuring the regression head can learn the underlying geometry robustly, regardless of the spatial downsampling factors applied earlier in the pipeline.

### 3.3.6 Chunking for High-Throughput I/O

Training the model for 150 epochs is only feasible if the BOLT pipeline is executed once rather than on every epoch. A naive dataloader would stream the raw h5 and npy files in whatever segment fits in memory and re-run the full pipeline (signal conditioning, voxelisation, downsampling, label alignment) on each pass, multiplying the preprocessing cost 150-fold and starving the GPU while the CPU works through every voxelisation and denoising call.

The chunking strategy moves the preprocessing out of the training loop entirely. The BOLT pipeline is executed once, offline, and the resulting spatiotemporal voxels and aligned label tensors are serialised to disk as contiguous 250-sample chunks (compressed `.pt`), indexed by a global `manifest.json` byte-offset map. During training, the dataloader reads these precomputed chunks directly into VRAM, so the 150 epochs execute against structured tensors rather than raw events.

#### LRU Caching in `VoxelDiskDataset`

To further amortise disk I/O, the custom `VoxelDiskDataset` wraps the chunk layout with a Least Recently Used (LRU) memory cache sized to hold the full 40-chunk dataset in the node's 120 GB RAM. During Epoch 1, chunks are read from NFS into memory as the dataloader encounters them, populating the cache in full. Every subsequent epoch is then served entirely from RAM, so the training loop never touches disk after Epoch 1. The LRU eviction policy is a safety net for cache-smaller-than-dataset configurations: the least-recently-used chunk is evicted to make room for the next load, ensuring the hot data is always ready for the GPU.

The combined effect of offline precomputation, chunked serialisation, and in-memory caching reduces the 150-epoch wall time from a projected multi-day run to approximately six hours at $\sim$160 ms/iter.

### 3.3.7 BOLT Pipeline: Visualized

Figure 3.2 details the order of which the aformentioned preprocessing steps are applied to the raw event data.

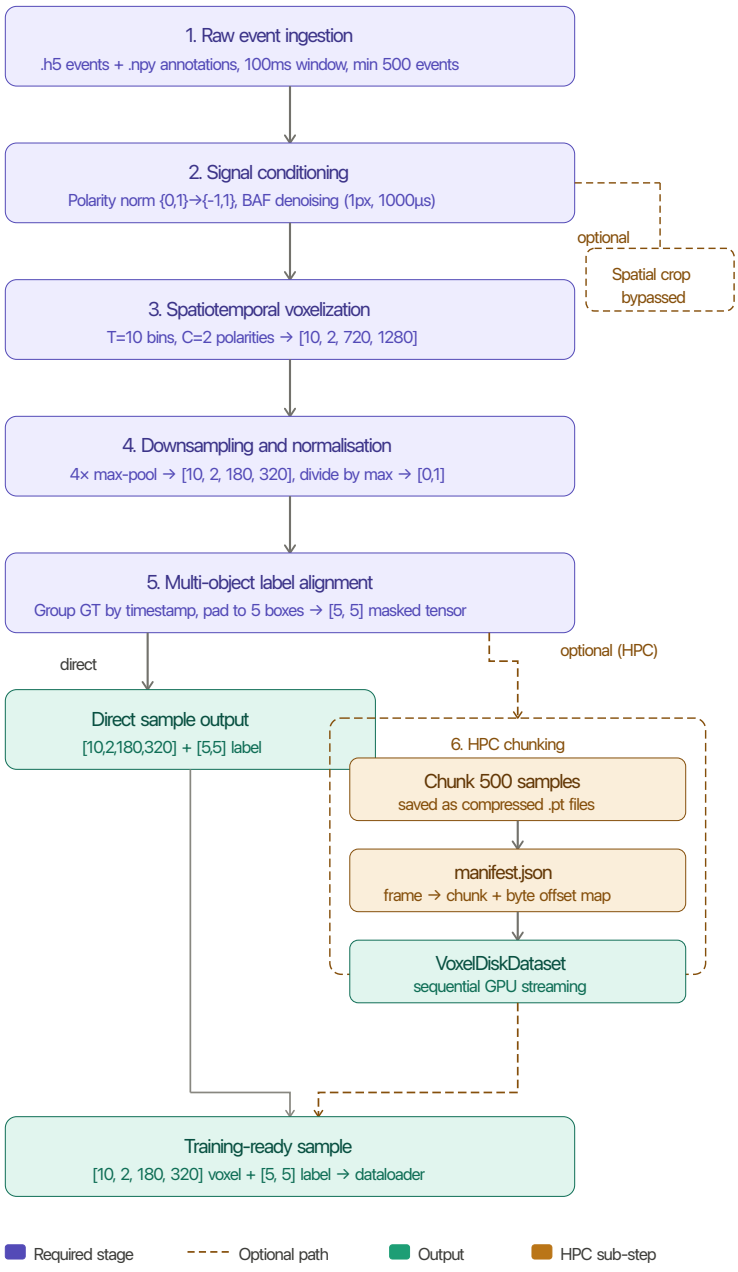



Figure 3.2: Overview of the spatiotemporal preprocessing pipeline applied to the ETraM dataset. Raw events are voxelized into a 4D tensor, spatially downsampled, and aligned with multi-object ground truth labels to produce a final sample of shape `[10, 2, 180, 320]` with a corresponding label tensor of shape `[5, 5]`.

## 3.4 Spiking Neural Network (SNN) Architecture

The core focus, effort and innovation of this project is the development and optimization of the `DetectorNX-G3-SNN` architecture. This model is explicitly designed to process high-definition neuromorphic event data while adhering to the computational paradigms of SNNs. By prioritizing event-driven, binary communication, the architecture aims to drastically reduce both the dynamic power consumption and the computational complexity compared to traditional deep learning models, making it superior for power and time constrained environments.

### 3.4.1 Spatiotemporal Input & Multi-Step Execution

The raw ETraM event data is initially preprocessed to a spatial resolution of $320 \times 180$ with $T = 10$ discrete temporal bins. Consequently, the network ingests a massive, high-dimensional spatiotemporal tensor of shape $[B, 10, 2, 180, 320]$ (Batch, Time, Polarity, Height, Width).

Given the high spatial dimensionality and the iterative nature of spiking neurons, native Python loop execution over the time dimension proved computationally restrictive. To overcome this bottleneck, the entire architecture was refactored to utilize the SpikingJelly framework's optimized Multi-Step (`step_mode='m'`) execution paradigm [11]; instead of iterating through the 10 time bins sequentially, the entire spatiotemporal tensor is passed directly into custom-compiled Compute Unified Device Architecture (CUDA) kernels. This low-level optimization enables high-speed, parallelized execution of the temporal dynamics, making the extensive benchmarking of this HD dataset feasible on local GPU hardware, and reducing overall training time from approximately 16 hours to just 7.

### 3.4.2 Neuron Dynamics: LIF

At the foundation of the architecture lies the LIF neuron model as defined in Section 2.3.1, which governs the temporal dynamics and binary spike generation of the network.

#### The Forward Pass (Spike Generation)

Both the membrane potential dynamics from Equation 2.3 and Heaviside spike function from Equation 2.4 are configured with the following project-specific parameters:

$$V_{th} = 0.2$$
$$V_{reset} = 0.0$$
$$\tau = 4.0$$

The voltage threshold $V_{th} = 0.2$ was tuned to balance information flow against noise. Earlier revisions used $V_{th} = 0.3$, which silenced the network, whereas decreasing $V_{th}$ further submitted erroneous background noise as spikes.

The time constant $\tau = 4.0$ creates a leaky effect, allowing the neuron to retain input evidence across roughly four to five timesteps – low enough to prevent over-saturation, but high enough for the signal to be remembered until the regression head before decay.

Following each spike, the neurons apply a hard reset to $V_{reset} = 0.0$, enforcing temporal sparsity by requiring the neuron to integrate new spatiotemporal information before re-activating, and providing the zero-centred baseline required for stable surrogate gradient descent [10].

#### The Backward Pass (Surrogate Gradients)

As described in Section 2.3.2, the Heaviside function's non-differentiable property poses a challenge in spiking neural networks. To get around the zeroed gradients and the consequent "dead neuron" problem, the SNN model employs the ATan function described in Section 2.3.2, where the backward pass approximates the step function using Equation 2.5; this mathematical substitution allows the network to learn complex spatiotemporal representations from scratch while maintaining a purely binary forward execution. In this implementation, SpikingJelly's default value of $\alpha = 2.0$ was used, which is the standard value used for high-performance SNN training.

### 3.4.3 Backbone: SEW-ResNet & Attention

To extract high-level semantic features and manage the computational complexity of the high-definition input, the architecture utilizes a deep residual backbone divided into five sequential stages, as outlined in Table 3.2.

| Stage | Name | Resolution | Channels | Primary Operation |
|---|---|---|---|---|
| 1 | Initial Stem | 90×160 | 64 | Stride-2 Conv + LIF (Downsamples HD input) |
| 2 | Backbone Alpha | 22×40 | 256 | Blocks 1 & 2 (SEWResBlocks with Stride-2) |
| 3 | Attention Bridge | 22×40 | 256 | Squeeze-and-Excitation (SE) Global Firing Recalibration |
| 4 | High-Res Backbone | 22×40 | 512 | Blocks 3 & 4 (SEWResBlocks with Stride-1) |
| 5 | Temporal Fusion | 22×40 | 512 | Summation of spikes across all $T$=10 timesteps |

Table 3.2: Detailed breakdown of the five sequential stages in the `DetectorNX-G3-SNN` backbone.

#### Spatial Downsampling

As illustrated, the architecture implements an aggressive 8× total spatial downsampling strategy to compress the initial $320 \times 180$ input. This is achieved through strided convolutions applied

within the Initial Stem and Backbone Alpha, progressively reducing the spatial dimensions to a highly compressed $22 \times 40$ grid while expanding the channel width up to 512.

### Spike-Element-Wise (SEW) Residual Blocks

Building deep SNNs presents significant challenges due to the "vanishing spike" problem inherent in standard residual connections. In a standard Residual Neural Network (ResNet), the residual addition occurs *before* the activation function. However, if we follow this standard and add two binary spike trains, the result is a non-binary, continuous value (e.g., $1 + 1 = 2$), which destroys the energy-efficient, event-driven nature of the network [10].

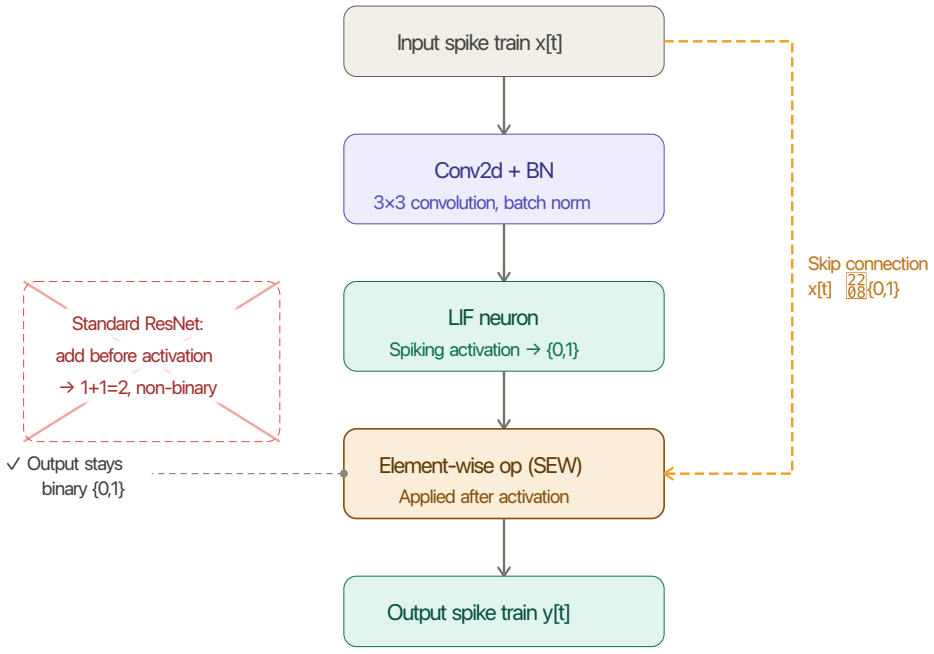



Figure 3.3: The SEW residual block (`SEWResBlock`) as proposed by Fang et al. [10].

To overcome this, the `DetectorNX-G3-SNN` architecture adopts the SEW residual paradigm [10], instantiated as `SEWResBlock` units throughout the backbone. Unlike standard Spiking ResNets, SEW ResBlocks apply the element-wise residual operation *after* the spiking activation function, guaranteeing that the output remains strictly binary. Fang et al. [10] prove that this reordering is both necessary and sufficient to achieve identity mapping in a purely spike-based network, resolving the degradation problem without sacrificing neuromorphic compatibility. In this architecture, this property is exploited to stack 4 SEWResBlocks across the backbone stages (Stages 2 and 4 in Table 3.2), enabling representational depth that would otherwise cause spike degradation under a standard residual formulation.

**Structural Attention: SE-Block**

In deep learning, a bottleneck refers to a point in the neural network where the information flow is forced through its most compressed and abstract representation. In this `DetectorNX-G3-SNN` architecture, the bottleneck exists at the transition point between stages 2 and 3 of the backbone, and represents two things:

1. **A Spatial Constraint (Resolution Limit):** At this stage, the original $320 \times 180$ spatiotemporal input has been downsampled by a factor of 8x via the strided convolutions in the stem and early residual blocks, resulting in a highly compressed $22 \times 40$ feature map. This spatial bottleneck is critical because each remaining "pixel" must now represent a large $8 \times 8$ receptive field of the original HD sensor data, forcing the network to discard fine-grained geometric redundancy while retaining the core structural boundaries of multiple detected objects.

2. **A Semantic Shift (High-Level Abstraction):** This juncture marks the definitive pivot where the network transitions from extracting low-level geometric primitives, such as spike-edge orientation and motion direction, to synthesizing complex semantic concepts. Without an explicit selection mechanism, the network at this depth is susceptible to noise and artifacts from the HD event stream, which can blur the distinction between legitimate targets and background interference.

To both mitigate and solve these bottleneck constraints, a Squeeze-and-Excitation (SE) mechanism is introduced in Stage 3 of the backbone, sitting directly at the transition point [17]. The SE block operates by performing a global average pooling of the 256 feature channels (the "Squeeze" phase), followed by two fully connected layers that calculate a weighting factor for each channel (the "Excitation" phase). This effectively mitigates the spatial constraint by allowing the network to incorporate global context into its local representations, compensating for the loss of resolution.

When applied in the context of an SNN, this acts as a dynamic firing modulator: the backbone's activations are re-calibrated by looking at the global context of the entire frame, magnifying the most informative channels whilst suppressing those containing artifacts. By applying this recalibration, the SE block ensures that only the most relevant "Objectness" (the probability that a specific region of an image contains a generic object of interest rather than background noise) features are passed into the final high-capacity blocks, effectively mitigating the semantic ambiguity before the final bounding box regression.

### 3.4.4 Firing Rate Regularization (FRR)

A common failure mode in deep SNN training is network silencing (zero spikes emitted) or over-saturation (firing continuously at every timestep), both of which halt gradient flow and

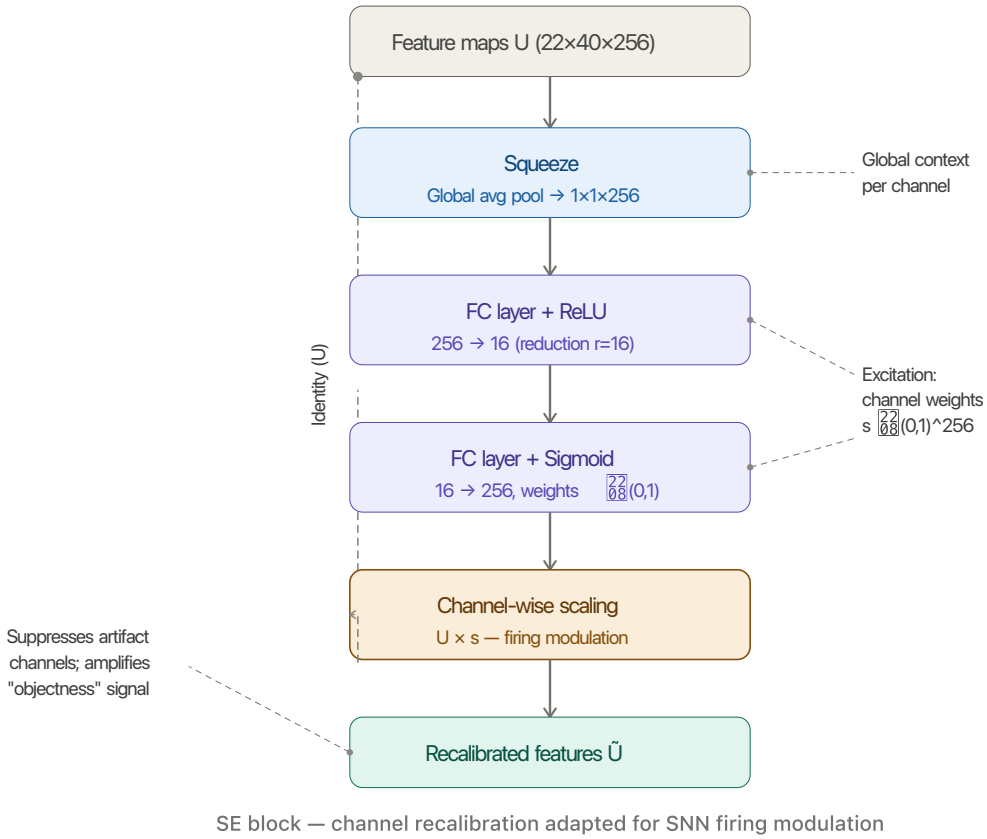



Figure 3.4: The Squeeze-and-Excitation (SE) block [17] as applied in Stage 3 of the `DetectorNX-G3-SNN` backbone ($22\times40\times256$).

destroy energy efficiency. To constrain the network, a custom Firing Rate Regularization (FRR) Metabolic Penalty was integrated into the training pipeline.

The FRR applies a squared error loss to the network's average firing rate:

$$L_{frr} = \lambda \cdot (Rate_{avg} - Target)^2 \tag{3.1}$$

This biologically inspired metabolic constraint [30] forces the 11.7M parameter backbone to maintain a selective but informative firing rate (e.g., $Target = 20\%$) across its hidden layers. Penalising derivation from a target firing rate is an established technique for preserving activation sparsity in directly trained SNNs [5, 41]. Adding this penalty to the model's loss ensures high information flow while strictly preserving the activation sparsity required for efficient neuromorphic deployment.

### 3.4.5 Multi-Box Regression Head

Following the final backbone stage, the fused spatiotemporal feature map is routed into a unified detection head structured as a three-stage pipeline: a channel projection, a dual adaptive pool, and a final regression mapping.

A $1 \times 1$ convolution first projects the 512-channel fused tensor down to 128 channels, compressing feature dimensionality ahead of the spatial reduction and lowering the parameter count of the downstream head.

The projected map is then passed through two parallel $8 \times 8$ adaptive pools, an Adaptive Average Pool and an Adaptive Max Pool, whose outputs are summed element-wise. Average pooling captures the global scene statistics but tends to smooth out sparse, informative responses; max pooling preserves the strongest localized activations but discards distributional context. Summing the two retains both signals, mitigating the spatial ambiguities observed during the iterative design process (detailed in Section 3.6).

The resulting $[B, 128, 8, 8]$ tensor is then mapped via final convolutional and linear layers to an output tensor of shape $[B, 5, 5]$, where $B$ represents the batch size. The final two dimensions encode the top 5 predicted bounding boxes globally per image, with each box defined by a feature tuple $\mathcal{F} = \{x, y, w, h, c\}$. Here, $(x, y, w, h)$ represent the normalized spatial coordinates and dimensions of the bounding box, and $c$ denotes the object confidence logit. This formulation allows the network to predict multiple vehicles simultaneously, which are subsequently evaluated using a vectorized Multi-Box Loss function.

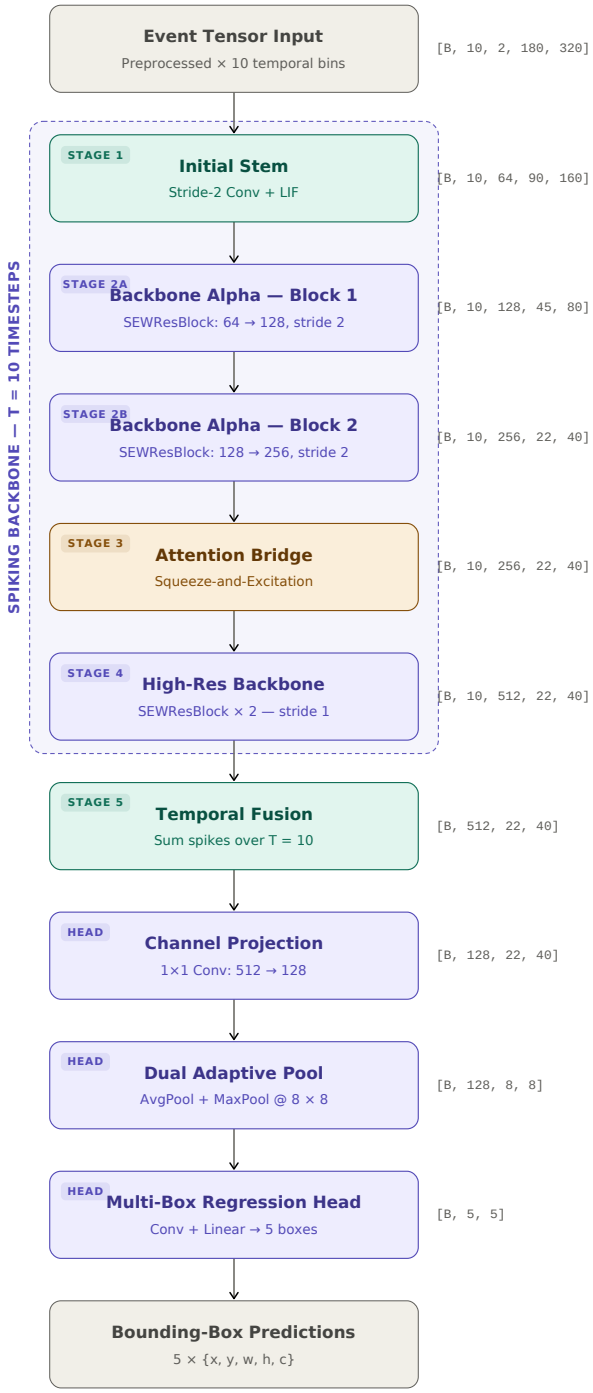



Figure 3.5: End-to-end data flow through the `DetectorNX-G3-SNN` architecture. The spiking backbone (Stages 1–4) processes the full spatiotemporal tensor in parallel across $T = 10$ timesteps, after which Temporal Fusion collapses the time dimension prior to the multi-box regression head. A full list of architectural and training hyperparameters is provided in Table A.1 of the appendix.

## 3.5 Convolutional Neural Network (CNN) Baseline

The CNN architecture `DetectorNX-G3-CNN` was engineered as the primary baseline to evaluate the SNN against. It is a 11.73M parameter CNN specifically designed to mirror the SNN architecture, satisfying the "Architectural Parity" functional requirement specified in 3.1.1. This ensures that the only variable being tested in this comparative study is the neural activation mechanism (event-driven binary spikes versus continuous-valued floats), making the comparison immune to common criticisms regarding unmatched or unoptimized baselines.

### 3.5.1 Structural Configuration & Layer-Wise Mapping

The CNN mirrors the SNN's configuration strictly 1:1, replacing every temporal and spiking component with a standard continuous equivalent. Table 3.3 outlines this direct mapping.

| SNN Component | CNN Equivalent | Technical Purpose |
|---|---|---|
| Spatiotemporal Voxel `[B, 10, 2, H, W]` | Flattened Channel Input `[B, 20, H, W]` | Translates 10 time bins into a format the respective network can ingest. |
| LIF Neuron (Leaky Integrate-and-Fire) | ReLU Activation | Provides the nonlinearity; SNN uses discrete pulses, CNN uses continuous values. |
| Binary Spikes $\{0,1\}$ | Continuous Floats $[0,\infty)$ | Represents the information "currency" being passed between layers. |
| Surrogate Gradient | Exact Derivative (Standard) | Enables backpropagation through the non-differentiable spiking threshold. |
| SEWResBlock (Spiking Shortcut) | Standard Residual Block | Implements identity mapping to prevent vanishing gradients during training. |
| Multi-Step Mode (`step_mode='m'`) | Single spatial pass | Processes the temporal window efficiently on the GPU. |
| Squeeze-and-Excitation (Spike-based) | Squeeze-and-Excitation (Float-based) | Recalibrates channel importance based on global frame context. |
| Temporal Summation ($\sum_{t=1}^{T} S_t$) | N/A (Direct Feature Pass) | Aggregates 100ms of evidence into a single map for the regression head. |
| Multi-Box Head (5 Boxes) | Multi-Box Head (5 Boxes) | Predicts coordinates and objectness scores for 5 targets simultaneously. |
| Synaptic Operations (SOPs) | Floating Point Ops (FLOPs) | The metric used to quantify and compare hardware energy efficiency. |

Table 3.3: 1:1 Architectural Mapping (SNN vs. CNN)

### 3.5.2 Input Strategy: Channel-Wise Temporal Integration

Unlike the SNN, which processes event data sequentially over discrete time steps $T$, the CNN baseline utilizes a flattened temporal input strategy. Instead of processing 10 timesteps sequentially, it concatenates the $T = 10$ bins (each containing 2 polarity channels) into a single input tensor of 20 channels, changing the input shape from $[B, 10, 2, H, W]$ to $[B, 20, H, W]$.

This approach allows the CNN to correlate events across the entire 100ms window in a single spatial pass. This channel-wise stacking of event bins is a standard and effective methodology for feeding asynchronous event-camera data into traditional, non-spiking CNN architectures [13].

### 3.5.3 Backbone: Identity Mapping & Residual Connections

The CNN backbone employs deep residual learning (Section 2.2), utilizing four Standard Residual Blocks that expand to 512 channels in the final stage. To satisfy Architectural Parity, the `SEWResBlock`s of the SNN were replaced with Standard Residual Blocks of the structure Conv $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Conv $\rightarrow$ BN + Shortcut $\rightarrow$ ReLU; these shortcut paths mitigate vanishing gradients and enable stable training at this depth [15].

### 3.5.4 SEWResBlock vs Standard ResBlock

The core mathematical difference between the two architectures lies in their summation logic and the resulting information flow across the shortcut connection:

- **Standard ResBlock (CNN):** Performs an accumulative floating-point addition ($y = f(x) + x$). The continuous intensity of the residual path is directly added to the identity path before the final ReLU activation. Here, the shortcut carries the *raw input features* forward. This mechanism allows the model to learn the "difference" (the residual) needed to improve the representation, focusing on information preservation [15].
- **SEWResBlock (SNN):** Performs a relational element-wise spiking operation *after* the activation function. Because adding two binary spikes (e.g., $1 + 1 = 2$) produces a non-binary value, the SNN uses a spike-merging function (such as ADD or AND) to preserve the discrete $\{0, 1\}$ state. Crucially, the SNN shortcut carries the *membrane history* forward, prioritizing temporal integrity.

Table 3.4 summarizes these foundational differences in identity mapping.

| **Feature** | **Standard ResBlock (CNN)** | **SEWResBlock (SNN)** |
|---|---|---|
| Summation | Accumulative ($y = f(x) + x$) | Relational (Spiking Element-Wise) |
| Output Type | Continuous Float $[0, \infty)$ | Binary Spikes $\{0, 1\}$ |
| Primary Benefit | Prevents vanishing gradients | Prevents spike vanishing |
| Mathematical Goal | Information preservation | Temporal integrity |

Table 3.4: Comparison of identity mapping between Standard ResBlocks and SEWResBlocks.

This distinction is particularly relevant for high-speed neuromorphic data: while the Standard ResBlock accumulates continuous values that effectively blur fast movements into a single averaged float over the 100ms window, the SNN equivalent preserves exact spike timing, maintaining the temporal precision required for the regression head.

### 3.5.5 Attention Mechanism: SE-Block

The CNN encounters the same bottleneck as its SNN counterpart (Section 3.4.3) and adopts the same SE mechanism [17], differing only in that the squeeze operates over continuous activations via standard global average pooling and the excitation produces a float-valued per-channel scale rather than modulating spike rates. The block confers no hardware-level sparsity benefit in this context; it is retained purely to preserve Architectural Parity with the SNN baseline (Section 3.1.1).

### 3.5.6 The "Gold Standard" Upper Bound

By constructing this custom CNN mirror, the baseline serves as the upper theoretical bound of this specific architecture's capability on the ETraM dataset. For example, if the CNN baseline achieves a Mean IoU of 48%, this represents the maximum performance extractable by this parameter configuration. If the corresponding SNN achieves 43% IoU, the 5% gap explicitly represents the "quantization cost" of switching from 32-bit continuous floats to 1-bit binary spikes [3]. Adhering to this methodology ensures that the evaluation of the neuromorphic paradigm is both fair and scientifically sound.

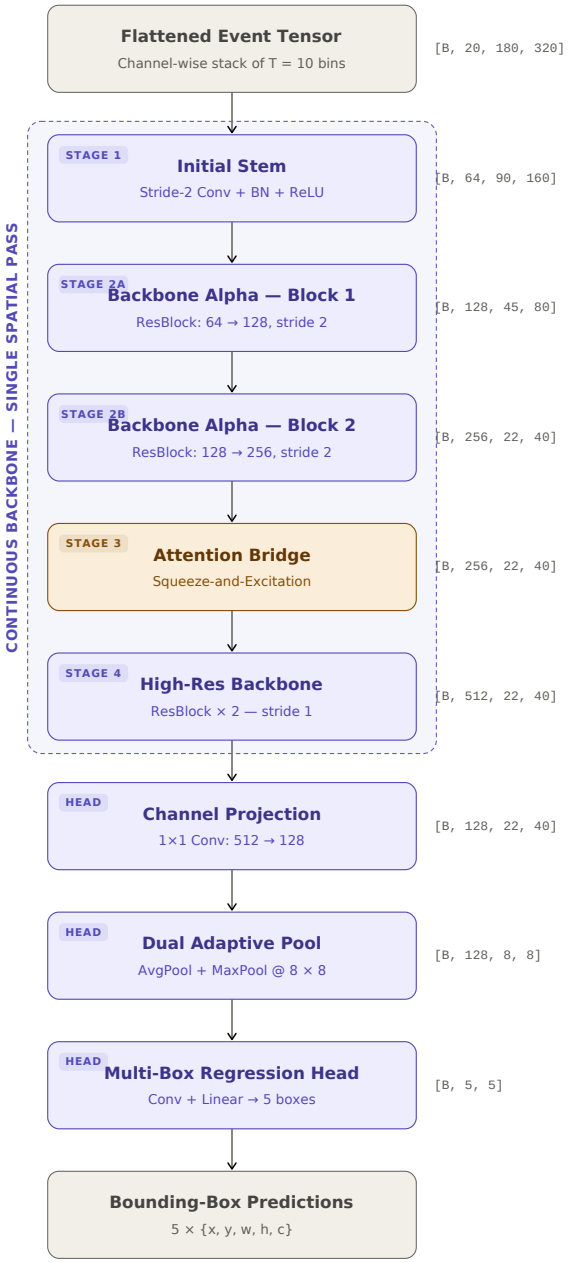



Figure 3.6: End-to-end data flow through the `DetectorNX-G3-CNN` baseline. The four backbone stages mirror those of the SNN (Figure 3.5) 1:1, differing only in activation (ReLU instead of LIF), residual formulation (Standard ResBlock instead of `SEWResBlock`), and temporal handling: the $T = 10$ bins are channel-stacked into a 20-channel input and processed in a single spatial pass, removing the need for a Temporal Fusion stage. A full list of architectural and training hyperparameters is provided in Table A.2 of the appendix.

## 3.6 Architectural Evolution

The final `G3` revision of the `DetectorNX` architecture was not the result of a single design phase, but rather the culmination of an iterative engineering process across four distinct generations. According to the project's success criteria, the model required both high temporal precision and robust spatial localization. This section documents the empirical obstacles encountered during development, and the actionable insights that led to the final optimized configuration.

### 3.6.1 Phase 0: The Spatial Cropping Pitfall (Early Prototyping)

Prior to the implementation of downsampling in the BOLT pipeline, early prototyping explored the use of aggressive spatial cropping to bypass the computational costs of high-definition event processing.

#### Architectural Intent & Technical Logic

The primary objective of this phase was to establish a high-resolution localized baseline by cropping the raw $1280 \times 720$ ETraM sensor data into a $224 \times 224$ patch, as illustrated in Figure 3.2; these patches were deterministically anchored to the centroid of the ground-truth vehicle annotation.

The technical logic behind this approach was rooted in improving the Signal-to-Noise Ratio (SNR) at the network's input. By utilizing *localized foveation*, we hypothesized that the initial stem layers would receive a significantly higher frequency of spikes per pixel for the target object. This increased spike density was intended to facilitate the extraction of fine-grained geometric features which are often attenuated or lost during global spatial downsampling, whilst also making it easier for the network to learn the features of vehicles by enlarging and zooming in on them. Additionally, since the localized patches occupied a smaller memory footprint, they could be processed without the need for aggressive pooling, and preserve the original sensor resolution.

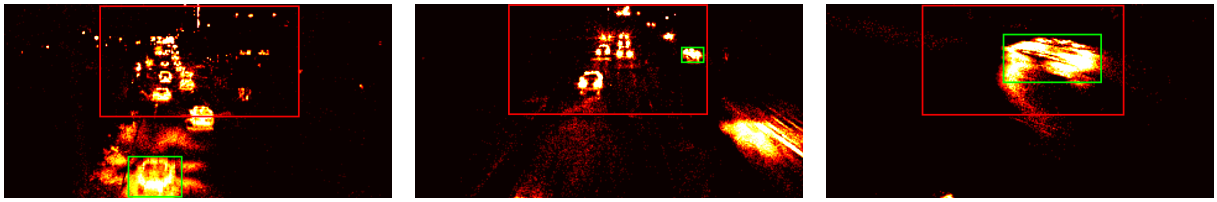
#### Empirical Obstacle: Constant Target Bias

During initial training cycles, the model achieved near-instantaneous convergence and exceptional accuracy metrics. However, a critical failure in the methodology was identified during validation: the model had failed to learn any meaningful feature representation.

Because the pipeline utilized deterministic centroid-anchored cropping, the target vehicle occupied the geometric center of every training sample. Consequently, the SNN succumbed to *Constant Target Bias*—a severe form of spatial overfitting where the regression head ignores the visual event stream and instead learns to output a constant coordinate vector of approximately $[0.5, 0.5, w, h]$. This eliminated the localization signal entirely; the model stopped "searching" for the vehicle and simply memorized the center coordinates. Such a model is fundamentally

useless for real-world traffic monitoring where vehicle positions are stochastically distributed across the full field of view, and are not pre-known to use spatial cropping before inference..

Critically, this collapse was invisible to the IoU metric; the validation split inherited the same centroid-anchored cropping as the training set, so the score was computed against a distribution that still rewarded the shortcut and reported no drop in accuracy. The bias only became apparent later in development, when inference was run on raw, full-frame input that bypassed the centroid crop — as shown in Figure 3.7, the predicted bounding box sits near the image centre regardless of where the vehicle actually lies.



(a) Train sample, IoU= 0.000. (b) Train sample, IoU= 0.015. (c) Test sample, IoU= 0.214.

Figure 3.7: Qualitative evidence of *Constant Target Bias* during Phase 0. Across three raw full-frame samples drawn from the training and test splits, the SNN regression head (red) emits a near-identical large central bounding box while the ground-truth vehicle (green) varies in position and scale between scenes. The marginally higher overlap in (c) arises only because the target vehicle incidentally sits near the image centre where the predicted box anchors, confirming the model has learned a constant coordinate shortcut rather than extracting vehicle features from the event stream.

#### Actionable Insight

This phase provided a critical methodological lesson: for a spiking architecture to learn robust spatial localization and objectness, it must be exposed to targets moving dynamically relative to the sensor boundaries within a full-frame (or semi-full-frame) field of view. This insight led to the total abandonment of centered cropping in favor of the global $4\times$ spatial downsampling strategy utilized in the final BOLT pipeline, ensuring that the model learned to correlate visual spike patterns with absolute spatial coordinates.

### 3.6.2 Generation 1 (G1): The Multi-Box Paradigm Shift

Following the resolution of the spatial cropping bias, the architecture progressed to full-frame processing. However, this transition immediately exposed fundamental limitations in the network's predictive capabilities regarding multi-object scenes.

#### Architectural Intent & Technical Logic

The primary objective of this phase was to develop a *scene-aware detector* capable of identifying multiple vehicles simultaneously within the ETraM traffic environment. To achieve this, the

network's output topology underwent a paradigm shift, transitioning from a simplistic single-box regression head (which output a single $[x, y, w, h]$ vector) to a Multi-Box Regression Head with a fixed-capacity output of $[B, 5, 5]$, where there are 5 features $(x, y, w, h, mask)$ and 5 bounding boxes predicted per inference. The technical logic dictated that the network required a mechanism to independently evaluate distinct spatial regions. To facilitate this, each of the five predicted boxes was augmented with a fifth dimension: an "Objectness" or Confidence Score. This structural addition allowed the model to not only regress bounding box coordinates but also to gate its own predictions, learning to suppress background artifacts with low confidence while maintaining high confidence for legitimate structural targets.

**Empirical Obstacle: Single-Target Ambiguity & Label Noise**

Initial prototypes utilizing the standard single-box regression head proved fundamentally incompatible with real-world traffic scenes. When multiple vehicles occupied the field of view simultaneously, the model faced a logical contradiction.

For instance, if a scene contained both Car A and Car B, the SNN backbone would naturally extract features for both. However, if the training label only provided the coordinates for Car A, the loss function would heavily penalize the network's correct detection of Car B as an error, introducing severe label noise. Over thousands of iterations, the model attempted to minimize this ambiguous loss by mathematically averaging the positions of all visible vehicles. This catastrophic failure mode resulted in a predicted bounding box that hovered uselessly in the empty space between vehicles, or a singular "super-box" attempting to encompass the entire roadway, leading to a collapse in IoU performance.

Insert images of the super box.

**Actionable Insight: The Vectorized Matching Loss**

This phase highlighted that resolving spatial ambiguity required a sophisticated mechanism to handle the *Assignment Problem*—the mathematical challenge of determining which of the 5 predicted boxes should be compared against which ground truth vehicle.

The actionable insight was the development of a Vectorized Multi-Box Loss. This computational innovation calculates a 3D matrix of all possible IoU overlaps between the 5 predictions and the $M$ ground truth objects in every batch. Utilizing greedy matching, the loss function identifies the best match for each real vehicle in the scene. Only this matched predicted box receives a *coordinate gradient*, signaling it to move closer to the target.

Crucially, all unmatched predicted boxes are assigned a target confidence of 0.0, acting as a metabolic penalty for false positives. This forces the model to learn that predicting a vehicle where none exists carries a heavy mathematical penalty, effectively cleaning up the SNN's vision and reducing the Ghost Detections prevalent in early iterations.

By transitioning to a multi-box architecture, we moved the SNN from a simplistic regression model to a sophisticated scene-aware detector, and introduced the Multi Box Label Alignment step shown in figure 3.2 into the BOLT pre-processing pipeline. This shift allowed the network to resolve the spatial ambiguity of multi-vehicle traffic, transforming the contradictory Label Noise of previous versions into a structured multi-target learning signal. This evolution was essential for achieving stable convergence on the high-definition, high-density event streams of the ETraM dataset.

### 3.6.3 Generation 2 (G2): The Expanded Spatial Head (7.00M Parameters)

While Generation 1 successfully established the multi-box logic, empirical validation revealed a persistent failure mode characterized by ghost Detections and overlapping predictions.

#### Architectural Intent & Empirical Obstacle

The primary objective of this phase was to resolve *Information Congestion* within the regression head. Generation 1 architecture utilized an Adaptive Average Pooling layer of $4 \times 4$, compressing the entire field of view into just 16 spatial blocks before entering the final linear layers.

This severe spatial bottleneck meant that when vehicles were in close proximity, or situated near clusters of sensor artifacts, their respective spiking features were averaged into an identical $4 \times 4$ block. The regression head lacked the spatial memory required to distinguish valid target spikes from background noise. Consequently, the model frequently outputted overlapping bounding boxes for a single vehicle or failed to suppress high-confidence noise, resulting in an inflated average of 2.50 detections per frame - significantly higher than the ground truth average of 2.02.

#### Actionable Insight: Quadrupling Spatial Memory

To resolve this congestion, the Adaptive Pooling resolution was expanded from $4 \times 4$ to an $8 \times 8$ grid, quadrupling the available spatial memory to 64 distinct blocks, providing the linear regression head with a substantially finer structural map of the backbone's activations.

This architectural refinement enabled localized Object-Noise Discrimination: with 64 distinct blocks, the model could successfully isolate the structured spike trains of a vehicle from the unstructured noise of adjacent sensor artifacts. This resulted in dramatic Confidence Polarization: max confidence scores for real objects surged to 0.99, while background noise was successfully pushed toward 0.0.

Consequently, the average detections per frame dropped to 2.06, nearly perfectly matching the ground truth. However, while the model's selectivity improved drastically, the Mean IoU plateaued at 41%. This confirmed that while the regression head now possessed sufficient capacity, the backbone's heavily downsampled resolution remained the fundamental bottleneck preventing higher geometric precision.

### 3.6.4 Generation 3 (G3): The High-Resolution Backbone (11.7M Parameters)

Following the expansion of the spatial head in Generation 2, the model demonstrated excellent selectivity but remained computationally constrained by its geometric precision.

#### Architectural Intent & Empirical Obstacle

The primary objective of this final phase was to break the Precision Ceiling established in Generation 2. Despite the regression head's capacity to isolate targets, the Mean IoU remained stagnant at 41%.

The fundamental obstacle was the severe spatial compression occurring within the backbone. By the final layer, the feature map had been downsampled to an $11 \times 20$ resolution. At this scale, a single pixel in the final feature map represented a massive $16 \times 16$ area of the original HD sensor. Consequently, if a vehicle's bounding box edge shifted by several pixels in the real world, the network was entirely blind to the change; the G2 generation simply lacked the geometric detail required to draw tight, pixel-perfect bounding boxes.

Furthermore, an analysis of the network's information flow revealed that the Generation 2 backbone was suffering from neural silencing, exhibiting an average firing rate of $< 1.5\%$. This extreme sparsity starved the regression head of the necessary temporal evidence required for high-confidence predictions.

#### Actionable Insight: Semantic Expansion & Metabolic Awakening

To overcome this precision ceiling, a multi-faceted architectural overhaul was implemented to create the final G3 configuration:

- **Resolution Boost & Semantic Depth:** To provide the necessary geometric detail, the convolution stride in Block 3 was reduced to $s = 1$. This effectively halted the downsampling earlier in the network, doubling the final feature map resolution to $22 \times 40$ (quadrupling the available spatial cells to 880). To handle the increased complexity and high-frequency sensor artifacts exposed by this higher resolution, the backbone's semantic was expanded via the addition of a 4th `SEWResBlock` (512 channels). This allowed the network to think deeper about the structural composition of the vehicle.

- **Metabolic Regularization:** To wake up the silent backbone and maximize information flow, an aggressive metabolic strategy was deployed. The LIF firing threshold was lowered drastically ($V_{th} = 0.5 \rightarrow 0.2$), making it significantly easier for neurons to activate. Concurrently, the FRR penalty multiplier ($\lambda$) was quintupled ($1.0 \rightarrow 5.0$) with a target rate of 20%. This forced the network to generate a dense, high-frequency stream of geometric data, providing the regression head with the robust temporal evidence needed to sense the exact boundaries of the vehicles.

- **Hardware Optimization:** Expanding the parameter count to 11.7M and quadrupling the spatial resolution exponentially increased the computational load. To prevent catastrophic latency penalties, the architecture was refactored to utilize SpikingJelly's `step_mode='m'` (*Fused-Kernel Optimization*) [11], which bypassed the Python interpreter to execute the temporal dimension ($T = 10$) directly on the GPU. This was coupled with AMP utilizing 16-bit tensor cores, yielding a $\sim 1.8\times$ training speedup and making the G3 configuration computationally viable.

Generation 3 successfully orchestrated the transition from basic structural detection to high-fidelity geometric precision. By halting the spatial degradation and enforcing strict metabolic regularization, the G3 model demonstrated that SNNs can achieve precise localization on HD event data without the severe latency penalties typically associated with deep spiking architectures.

### 3.6.5 Generations: Summarized

To summarize, Table 3.5 shows the evolution of the `DetectorNX` architecture across the three generations.

| **Generation** | **Primary Architectural Shift** | **Params** | **Main Strength** | **Final Limitation** |
|---|---|---|---|---|
| G1 | Multi-Box Head $[B,5,5]$ | 5.43M | Handled multi-vehicle scenes simultaneously | High false positives in low-density regions ("Ghosting") |
| G2 | 8×8 Spatial Pooling Head | 7.00M | Solved object-noise ambiguity through improved spatial aggregation | 41% IoU ceiling due to insufficient spatial resolution |
| G3 | 22×40 Resolution with additional SEWResBlock (Stage 4) | 11.72M | High-fidelity edge precision through full-resolution backbone processing | Higher computational metabolic cost in SOPs |

Table 3.5: Generational evolution of the `DetectorNX` architecture.

## 3.7 Training Structure

This section specifies how the `DetectorNX-G3-SNN` architecture established in Section 3.4 is trained on the precomputed voxel chunks produced by the BOLT pipeline (Section 3.3). Each hyperparameter choice addresses a specific failure mode encountered during architectural evolution (Section 3.6), and the configuration summarised here is the one used to train the model benchmarked in Section 3.8.

### 3.7.1 Objective Function

The total training loss combines a vectorised multi-box loss with the firing rate regularisation introduced in Section 3.4.4:

$$L_{total} = L_{coord} + L_{conf} + L_{frr} \tag{3.2}$$

The multi-box loss operates on the $[B,5,5]$ head output. A broadcast IoU matrix of shape $[B,K,M]$ is computed between the $K = 5$ predicted boxes and the $M = 5$ padded ground-truth slots, and each ground-truth slot is independently paired with its highest-IoU prediction via `ious.argmax(dim=1)`. Each matched pair contributes a combined coordinate term

$$L_{coord} = \text{SmoothL1}(\hat{\mathbf{b}}, \mathbf{b}) + \lambda_{giou} \cdot (1 - GIOU(\hat{\mathbf{b}}, \mathbf{b})) \tag{3.3}$$

with $\lambda_{giou} = 2.0$. SmoothL1 supplies the per-coordinate gradient; the GIoU term contributes the geometric signal on disjoint boxes (Section 2.2.4). The confidence term applies `BCEWithLogitsLoss` to the raw logits, targeting 1.0 for the slot matched to each real object and 0.0 elsewhere. The fused sigmoid/BCE formulation is numerically stable under the float16 autocast described in Section 3.7.2.

The FRR term is a symmetric squared penalty across all nine spike tensors emitted by the backbone (the stem LIF output plus two per `SEWResBlock`):

$$L_{frr} = \frac{\lambda_{frr}}{N} \sum_{i=1}^{N} (r_i - r^*)^2 \tag{3.4}$$

with target rate $r^* = 0.20$ and $\lambda_{frr} = 5.0$. The spike tensors remain in the autograd graph, so FRR gradients propagate backward through the ATan surrogate gradient and adjust backbone convolution weights directly, making regularisation an active training signal rather than a post-hoc diagnostic.

### 3.7.2 Optimisation

Parameters are updated with Adam [21] at $\eta = 2 \times 10^{-4}$ and weight decay $10^{-4}$. The learning rate is controlled by `ReduceLROnPlateau` (factor 0.5, patience 15 epochs) operating on an

exponentially smoothed validation IoU:

$$\mathrm{EMA}_t = \beta \cdot \mathrm{EMA}_{t-1} + (1-\beta) \cdot \mathrm{IoU}_t, \qquad \beta = 0.9 \tag{3.5}$$

Scheduler steps are gated by EMA $\geq 0.10$, which prevents premature decay while the model is still below baseline localisation competence.

Training runs under `torch.amp.autocast('cuda')` with a `GradScaler`, yielding a $\sim 1.8\times$ throughput gain and halved activation memory on the NVIDIA A100 tensor cores. After loss scaling but before the optimiser step, gradients are unscaled and clipped to $\ell_2$ norm 1.0 to contain the heavy gradient tail produced by the eight-deep surrogate path through the four `SEWResBlock` pairs.

### 3.7.3 Validation and Checkpointing

After each training epoch the model is evaluated across the full validation split. Predictions are sigmoid-activated, real objects are identified by mask $> 0.5$, and a detection is counted as active when its confidence logit clears the same threshold. The per-epoch metric is the mean of the best IoU achieved against each real object across the active predictions. A best-model checkpoint is written whenever this raw IoU surpasses the current maximum; a final-epoch checkpoint is preserved unconditionally for ablation.

### 3.7.4 Hyperparameter Summary

Table 3.6 consolidates the trained configuration, including the values forward-referenced by the success-criteria traceability table (Table 3.1).

Table 3.6: Training configuration for the `DetectorNX-G3-SNN` production run.

| **Group** | **Parameter** | **Value** |
|---|---|---|
| Input | $T$ (timesteps) | 10 |
| | Batch size | 8 |
| Neuron (LIF) | $\tau$ | 4.0 |
| | $V_{th}$ | 0.2 |
| | Surrogate | ATan |
| Head | Adaptive pool | $8 \times 8$ |
| | Head channels | 128 |
| Loss | $\lambda_{giou}$ | 2.0 |
| | $\lambda_{frr}$ | 5.0 |
| | $r^*$ (target rate) | 0.20 |
| Optimiser | Adam $\eta$ | $2 \times 10^{-4}$ |
| | Weight decay | $10^{-4}$ |
| | LR factor | 0.5 |
| | LR patience (epochs) | 15 |
| | EMA $\beta$ | 0.9 |
| | IoU gate | 0.10 |
| Stability | Grad clip $\|g\|_2$ | 1.0 |
| | AMP | CUDA float16 |
| Training | Epochs | 150 |

## 3.8 Experimental Setup & Benchmarking

Having established the theoretical design and optimization of the `DetectorNX` architecture, both the SNN and its CNN counterpart, the final step is to formulate a rigorous evaluation strategy to empirically validate its performance against the continuous CNN baseline. In accordance with the project's primary objectives, the evaluation cannot be limited to a singular metric of bounding box accuracy; rather, it must holistically assess the neuromorphic paradigm. To achieve this, a comprehensive benchmarking suite was designed (as mentioned in Figure 3.1) comprising five distinct experimental protocols:

- Dynamic Power Estimation
- Quantization Error Analysis (Accuracy vs. Object Size)
- Confidence Calibration
- Temporal Evidence Accumulation
- Real-time Inference latency
- Theoretical Energy Consumption
- Global Evaluator: Mean/Median IoU, $mAP_{50}$, Recall, and Average Detections Per Frame

This multi-faceted methodology ensures that the comparative findings are statistically robust and firmly grounded in the established literature surrounding event-based vision.
Change last sentence.

### 3.8.1 Dynamic Power Estimation

A fundamental objective of this research is to quantify the energy efficiency gains achieved by transitioning from a continuous CNN to a bio-plausible SNN. Traditional CNN architectures are characterized by static power consumption; they perform a dense, fixed number of MAC operations for every forward pass, regardless of the information density within the input scene. In contrast, SNNs leverage event-driven dynamics, where metabolic energy is consumed only when a neuron reaches its firing threshold and generates a discrete spike.

The Dynamic Power experiment investigates the reactive scalability of the architecture by mapping instantaneous energy consumption against the temporal density of the input stream. This analysis serves to validate the event-driven nature of the SNN, demonstrating that its metabolic cost is a dynamic function of activation sparsity—mimicking biological neural efficiency by consuming energy only in response to informative stimuli.

### CNN Power Estimation

In a standard CNN, the total energy per frame ($E_{CNN}$) is calculated by multiplying the total number of Floating-Point Operations (FLOPs) by the cost of a 32-bit MAC ($E_{MAC}$):

$$E_{CNN} = \sum_{l=1}^{L} (FLOPs_l \times E_{MAC}) \tag{3.6}$$

Where $E_{MAC} = 4.6$ pJ per 32-bit floating point operation, as referenced in Table 3.7 [16].

The number of FLOPs is the total number of MACs operations required to produce the output feature map; in a CNN, this number is static, regardless of what the image contains. The FLOPs of a CNN can be calculated as follows:

$$FLOPs_l = (H_{out} \times W_{out}) \times (K \times K) \times C_{in} \times C_{out} \tag{3.7}$$

Where $(H_{out} \times W_{out})$ is the number of pixels in the output frame, $(K \times K)$ is the size of the convolutional kernel, $C_{in}$ is the number of input channels, and $C_{out}$ is the number of output channels.

### SNN Power Estimation

For the spiking architecture, the calculation is scaled by the activation sparsity ($\zeta$) of each layer. The metabolic cost of the spiking backbone can be quantified via the SOP metric; unlike the static complexity of the CNN baseline, the SOP count for a given layer is a dynamic function, which can be calculated as follows:

$$SOPs_l = FLOPs_l \times \zeta \times T \tag{3.8}$$

where $\zeta_l$ represents the average firing rate of layer $l$, $T$ is the number of temporal simulation steps ($T = 10$).

In order to obtain $\zeta_l$, a specialized `SpikeRateMonitor` tool was developed. The tool attaches a hook to every LIF inside the SNN backbone, which listens to the layer during the forward pass and intercepts the binary spike tensor before it moves to the next layer. For every layer $l$, the SNN produces a spike tensor $S_l$ with the shape $[T,B,C,H,W]$, where $T = 10$ is the number of timesteps, $B$ is the batch size, and $C,H,W$ are the spatial dimensions. Since a spike is binary (1 for fire, 0 for rest), $\zeta$ can be calculated by taking the global mean of the entire tensor:

$$\zeta_l = \frac{1}{N} \sum_{i=1}^{N} S_{l,i} \tag{3.9}$$

To calculate Dynamic Power, $\zeta$ was calculated by running inference over all validation samples and taking the mean firing rate per layer across all samples in the dataset.

Mention SpikeRateMonitor in Evaluation Suite Section and Diagram

Because SNNs utilize binary activations, the expensive multiplication part of the MAC operation is omitted, leaving only a AC of the weight to the membrane potential. The total energy per frame ($E_{SNN}$) is defined as:

$$E_{SNN} = \sum_{l=1}^{L} (SOPs_l \times E_{AC}) \tag{3.10}$$

where $E_{AC} = 0.9$ pJ is the energy cost of a single floating-point addition, as referenced in Table 3.7 [16].

### 3.8.2 Quantization Error Analysis (Accuracy vs. Object Size)

The Quantization Error Analysis experiment empirically tests the $1/T$ quantisation ceiling introduced in Section 2.3.3: it measures the extent to which the SNN's finite representational states limit regression precision on objects of differing spatial scales, compared against the continuous 32-bit CNN baseline which is not subject to this ceiling.

#### Core Metrics and Scale Partitioning

The primary performance metric for this experiment is the mIoU, which quantifies the overlap between the predicted bounding box ($P$) and the ground truth ($GT$). To analyze the impact of scale on regression precision, the validation set is partitioned into three distinct categories based on the absolute pixel area ($A_{px}$) of the ground truth objects:

- **Small:** Objects in the bottom 33rd percentile of area (typically distant vehicles or small artifacts).
- **Medium:** Objects in the middle 33rd percentile.
- **Large:** Objects in the top 33rd percentile (vehicles in close proximity to the sensor).

#### Mathematical Framework

The categorization and evaluation are governed by two fundamental geometric formulas. First, the pixel area is calculated by denormalizing the ground truth dimensions relative to the raw sensor resolution:

$$A_{px} = (W_{norm} \cdot W_{sensor}) \times (H_{norm} \cdot H_{sensor}) \tag{3.11}$$

where $W_{sensor} = 1280$ and $H_{sensor} = 720$. Subsequently, the precision of the model's spatial regression is quantified via the IoU formula:

$$IoU = \frac{\text{Area}(P \cap GT)}{\text{Area}(P \cup GT)} \tag{3.12}$$

#### Hypothesis: The Scale-Dependent Accuracy Gap

We hypothesize a significant scale-dependent accuracy gap between the two architectures. For large-scale objects, the SNN is expected to achieve parity with the CNN baseline: because the coordinate values are high, the minor fluctuations inherent in binary spike-trains represent a negligible percentage of the total object area, allowing for stable regression.

Conversely, for small-scale objects, a substantial performance divergence is anticipated. For the $T = 10$ configuration used here, this ceiling resolves to eleven discrete states $(0/10, 1/10, \ldots, 10/10)$. As documented by [3], this rounding error, or quantization penalty, is massive relative to the dimensions of small vehicles. If a target requires a precise coordinate gradient (e.g., 0.154) for accurate localization, the SNN is physically forced to discretize this value, establishing a fundamental "resolution floor" that limits mIoU on small features. While the use of `SEWResBlock` architectures improves signal flow [10], it does not inherently resolve this quantization of the final regression output, which is further exacerbated by the low event counts typical of small objects in event-based vision [12].

### 3.8.3 Confidence Calibration

The Confidence Calibration evaluates predictive reliability by investigating the calibration gap between spiking and continuous architectures. Modern CNNs are prone to systematic overconfidence, frequently assigning high objectness scores to inaccurate detections [14]. Conversely, the temporal dynamics of SNNs may offer superior uncertainty quantification, providing a more robust reliability profile for safety-critical automotive tasks [33].

#### Methodology: Reliability Diagrams

To quantify calibration, we generate a Reliability Diagram mapping predicted confidence ($C$) against actual regression accuracy ($IoU$) for every validation detection. In a perfectly calibrated system, these values align along the identity line ($C = IoU$). Clumping in the high-confidence, low-accuracy quadrant indicates an overconfidence bias, where the model fails to signal its own predictive errors.

#### Hypothesis: Spikes as Uncertainty Sensors

The CNN baseline is hypothesized to exhibit significant miscalibration due to its continuous activation functions and exact gradient descent, which tend to overfit the confidence head. The SNN is expected to demonstrate a more linear, calibrated trend. In the spiking domain, low-quality features generate fewer spikes, naturally leading to lower membrane potentials in the detection head. This mechanism allows the SNN to function as an implicit uncertainty sensor, where discrete spike rates effectively gate the output confidence logit relative to the strength of the temporal evidence.

### 3.8.4 Temporal Evidence Accumulation

The Temporal Evidence Accumulation experiment investigates the sequential decision-making dynamics inherent to the SNN architecture. Unlike the CNN baseline, which performs a single spatial pass over the fully integrated 100ms voxel, the SNN processes the event stream iteratively across $T = 10$ timesteps. This property makes SNNs intrinsically suited to low-latency spatiotemporal integration — the capacity to produce meaningful predictions before the full observation window has elapsed [27]. The experiment therefore examines whether this theoretical advantage manifests empirically: specifically, whether the model can produce a valid bounding box regression before the 100ms window is complete.

In the context of vehicle object detection, this is a highly significant experiment — for example, if the SNN can accurately show a bounding box at $t = 5$ (50ms) instead of the full $t = 10$ (100ms) as the CNN would, this would lead to a 50% reduction in detection time. In a time-sensitive domain such as vehicle and pedestrian detection, such advantages could be the difference between a crash and a safe stop.

#### Methodology: Temporal Snapshots

To evaluate the evolution of predictions, we extract intermediate outputs at three specific checkpoints: $t = 2$ (20ms), $t = 5$ (50ms), and $t = 10$ (100ms). For each snapshot, the candidate bounding box coordinates and instantaneous confidence scores are recorded to observe the *hunting behavior* of the regression head: the observable process where the networks' predicted bounding box moves, vibrates or searches for the object's true edges before finally settling into a stable, high-precision detection. This process is driven by the discrete LIF dynamics:

$$v_i[t] = v_i[t-1] + \frac{1}{\tau}\left(\sum_j w_{ij} s_j[t] - (v_i[t-1] - V_{reset})\right) \tag{3.13}$$

where $v_i[t]$ represents the accumulated evidence (membrane potential) and $\sum w_{ij} s_j[t]$ represents the incoming observations (spikes). This recursive filtering allows the network to update its belief about vehicle localization with every incoming pulse of data.

#### Hypothesis: Bounding Box Sharpening

We hypothesise a phenomenon of *Bounding Box Sharpening* over time, consistent with the rapid categorisation principle established in Section 2.3.3. At $t = 20$ms, we anticipate high coordinate variance, as the majority of LIF neurons will not have accumulated sufficient membrane potential to cross the firing threshold ($V_{th} = 0.2$) within the first two of ten timesteps. As $t$ progresses toward 100ms, continued spike train integration allows stochastic membrane noise to average out, consolidating responses to the vehicle's geometric edges into a stable coordinate estimate.

### 3.8.5 Real-Time Latency Benchmark

This experiment quantifies the forward-pass inference speed of both architectures on conventional GPU hardware at batch size $B = 1$, simulating a live single-stream event camera deployment. Latency (ms/f) and throughput (FPS $= 1000/$ms/f) are measured using `torch.cuda.Event` timing to capture true GPU execution time, bypassing Central Processing Unit (CPU)-to-GPU communication overhead.

#### Hypothesis

The CNN baseline is expected to exhibit lower latency, as GPUs are optimised for the dense matrix multiplications that underpin its single flattened spatial pass. The SNN must instead simulate LIF membrane dynamics iteratively:

$$v[t] = v[t-1] + x[t] - \beta \cdot v[t-1] \tag{3.14}$$

This constitutes a hardware-mismatch penalty: the latency advantage of SNNs only fully materialises on asynchronous neuromorphic hardware, not on synchronous GPU accelerators [4, 1]. The experiment aims to quantify this penalty and assess whether `step_mode='m'` parallelisation [11] brings the SNN within a $2\times$–$3\times$ latency margin of the CNN.

#### Limitations

The measured GPU latency figures cannot be extrapolated to neuromorphic silicon. The A100's 312 TFLOPS (FP16) characterises dense matrix throughput, which cannot equally be judged against asynchronous spike-based execution. While Loihi benchmarks report up to $10\times$ lower latency over embedded GPUs for batch-size-1 inference [4], these figures are task-specific; direct deployment on neuromorphic hardware is identified as future work.

### 3.8.6 Theoretical Energy Consumption

The Theoretical Energy Consumption is very similar in nature to the Dynamic Power experiment discussed in Section 3.8.1. It uses the same principles and formulae for SOPs and FLOPs, but differs in the final calculation: as supposed to calculating the dynamic power consumption per frame, this experiment aims to find the average power consumption per frame across the entire dataset, and extrapolates for multiple hardware platforms:

$$\bar{E}_{SNN} = \sum_{l=1}^{L} (SOPs_l \times E_{chip}) \tag{3.15}$$

$$\bar{E}_{CNN} = \sum_{l=1}^{L} (FLOPs_l \times E_{MAC}) \tag{3.16}$$

The calculation mirrors Equation 3.10, however $E_{SNN}$ replaces $E_{AC}$ for $E_{chip}$, which is the power consumed per SOP for different chipsets as depicted in Table 3.7.

Subsequently, the total percentage energy saving is formulated using the following equation:

$$\text{Energy Savings} = \left(1 - \frac{\bar{E}_{SNN}}{\bar{E}_{CNN}}\right) \times 100\% \tag{3.17}$$

**Energy Constants Per Operation ($E_{chip}$) Across Multiple Platforms**

| Architecture / Platform | Operation Type | Energy ($E_{chip}$) | Primary Citation |
|---|---|---|---|
| CNN Baseline | MAC | 4.6 pJ | [16] |
| Theoretical Silicon Limit | AC | 0.9 pJ | [16] |
| Intel Loihi 1 | SOP | 23.6 pJ | [4] |
| IBM TrueNorth | SOP | 26.0 pJ | [25] |
| NVIDIA DGX A100 (FP32) | FLOP | 19.6 pJ | [39] |

Table 3.7: Energy constants per operation ($E_{chip}$) for each architecture and platform.

**Hypothesis**

We hypothesize that the SNN will demonstrate a metabolic advantage of at least one order of magnitude ($> 90\%$) over the CNN baseline, predicated on two complementary properties of the $\bar{E}_{SNN}$ formulation:

1. **Activation sparsity:** By exploiting the sparsity of the ETraM event stream, the backbone is expected to maintain $\zeta < 5\%$, bypassing the majority of computations that the CNN performs densely regardless of scene activity.

2. **Mathematical simplification:** Replacing MAC operations with binary AC operations yields a $5\times$ reduction in per-operation energy cost (0.9 pJ vs 4.6 pJ), which is expected to provide a substantial efficiency dividend even when accounting for the $T = 10$ temporal overhead [16].

When extrapolated to physical neuromorphic platforms, the SNN's event-driven resting state is expected to preserve a power advantage over synchronous GPU accelerators despite the higher per-SOP costs of Intel Loihi (23.6 pJ) and IBM TrueNorth (26.0 pJ) compared to per-FLOP cost of the benchmark NVIDIA A100 (19.6 pJ) documented in Table 3.7 [4, 25, 39].

### 3.8.7 Global Performance Evaluator

The Global Performance Evaluator serves as the primary comparative evaluation between the SNN and CNN architectures, aggregating performance across the full 200-sample validation

set. Unlike the preceding targeted experiments, this benchmark establishes the *Performance-Efficiency Frontier*: quantifying the accuracy penalty incurred by transitioning to discrete spiking dynamics, whilst validating whether the multi-box refinements and others in the G3 revision of `DetectorNX` mentioned in Section 3.6 are sufficient to resolve scene-level detection ambiguity.

### Metrics

In this experiment, four complementary metrics are evaluated:

- **mIoU:** measures geometric precision between predicted and ground truth bounding boxes.
- **$\mathbf{mAP_{50}}$:** the COCO-convention detection-quality metric, measuring average precision at a fixed 0.5 IoU threshold; enables external comparison against published detectors and validates that mIoU parity is not an artefact of the overlap-only evaluation.
- **Recall:** measures the proportion of ground truth objects successfully localised at a confidence threshold of 50%, penalising architectures that fail to respond to sparse event regions.
- **Average Detections Per Frame (ADP):** quantifies detector selectivity relative to the ground truth mean of 2.02 objects per frame; significant positive deviation indicates spurious "ghost" detections arising from background noise misclassification.
- **Mean Prediction Confidence**: captures network self-certainty by averaging sigmoid-activated confidence scores across all positive detections, probing whether spiking dynamics produce decisive or hesitant class activations.

### Hypothesis

We hypothesise that whilst the CNN establishes the upper mIoU bound, the SNN will achieve *detection parity* in recall and ADP, yielding an average detection count consistent with the ground truth distribution. This would demonstrate that spiking temporal dynamics are sufficient for objectness discrimination without continuous-valued feature maps [30, 3]. A measurable confidence gap is also anticipated: the SNN's binary spike outputs are expected to produce lower but temporally stable confidence distributions across the $T = 10$ timestep window, consistent with prior observations that surrogate-trained networks distribute representational energy across timesteps rather than single forward passes [10].

### Significance

This experiment provides three contributions. It delivers the *parity proof*, framing the accuracy-efficiency tradeoff quantitatively. It offers *logical validation* that the Phase 1 ambiguity problem has been resolved through the architectural refinements described in Section 3.6. Finally,

evaluation across the full validation set ensures statistical robustness, confirming that results are representative of the broader ETraM environment rather than artefacts of favourable individual frames.

## 3.9 Results & Evaluation

Having established a rigorous, multi-faceted benchmarking methodology in Section 3.8, this final section presents the empirical evaluation of the G3 generation of the `DetectorNX` architecture against its CNN counterpart. The following results were conducted on the pre-processed `BOLT-10K-v2` dataset snapshot, using a validation split to ensure strict comparative integrity.

Rather than relying on a singular metric of spatial accuracy, the following analysis systematically deconstructs the performance-efficiency frontier of the neuromorphic paradigm: By sequentially examining global detection parity, energy consumption, quantization-induced scaling errors, and predictive calibration, this evaluation aims to quantify the precise trade-offs inherent in transitioning from 32-bit floating-point convolutions to 1-bit, event-driven spiking dynamics for high-definition automotive perception.

### Runtime Environment

The following experiments were collected using the following model and data configuration:

- **Model:** The G3 generation of the `DetectorNX` model architecture was used for both SNN and CNN experiments, as described in Section 3.6.
- **Dataset:** A stratified-sampling approach was used to sample the full ETraM dataset, in order to ensure that all scenes (day and night) were represented using the same proportions. For training and validation, $10,000$ samples were selected.
- **Precomputing:** The `BOLT-10K-v2` dataset snapshot was formed by applying the BOLT (Section 3.3) pipeline to the sampled $10,000$-sample dataset.
- **Training:** The `DetectorNX` model was trained on the `BOLT-10K-v2` dataset using an 80% training data split.
- **Evaluation:** The results of the following experiments were collected using a 200-sample subset from the validation split of the `BOLT-10K-v2` dataset, using the pre-trained `DetectorNX` model.
- **Compute:** The University of Manchester's HPC was used for pre-processing, training and evaluation. The environment was set up using 12 CPU cores, 120 GB RAM, and one NVIDIA A100 (80 GB) GPU.

### 3.9.1 Global Evaluator

This global overview defines the baseline from which all subsequent power and precision trade-offs are derived.

#### Quantitative Results

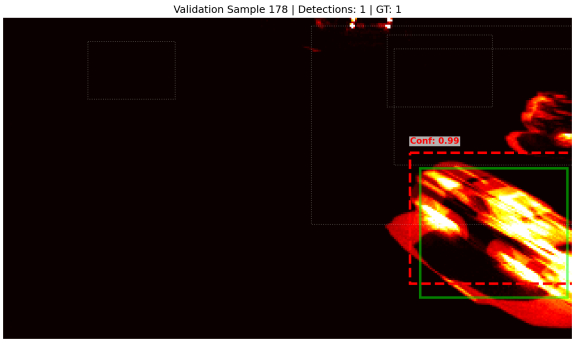
Table 3.8 summarizes the headline results for the G3 generation of the `DetectorNX` model architecture against its CNN counterpart.

| **Performance Metric** | **`DetectorNX-G3-SNN`** | **`DetectorNX-G3-CNN`** |
|---|---|---|
| Mean IoU | 0.4613 | 0.4861 |
| $mAP_{50}$ | 0.5977 | 0.6550 |
| Recall (Detections $> 0.5$) | 100.0% | 100.0% |
| Avg. Detections per Frame (GT) | 2.02 | 2.02 |
| Avg. Detections per Frame (Pred) | 1.73 | 1.75 |
| Max Confidence | 0.9930 | 0.9918 |
| Min Confidence | 0.0011 | 0.0132 |

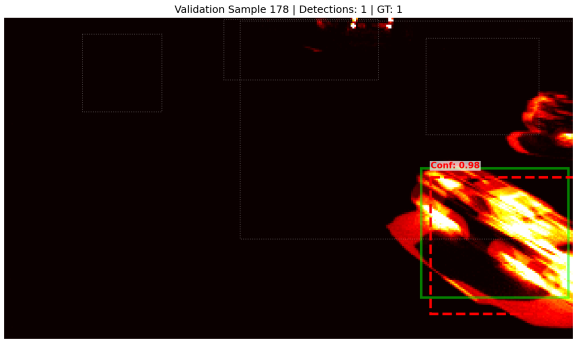
Table 3.8: Global Performance Benchmark results for `DetectorNX-G3-SNN` and `DetectorNX-G3-CNN`, evaluated across the full 200-sample ETraM validation set [40]. Ground truth average detections per frame is 2.02 for both models by definition.

#### Qualitative Results

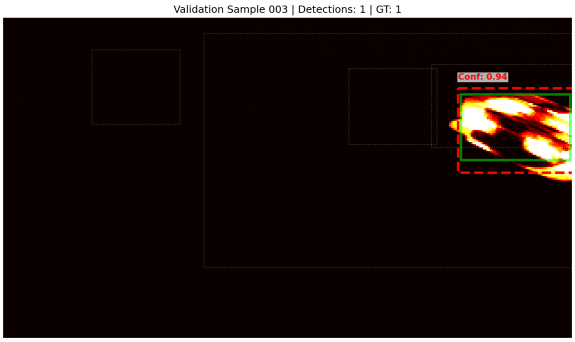
A qualitative audit was also conducted to visualize the models' spatial reasoning across the validation subset. Figure 3.8 illustrates the architectures' ability to resolve vehicle geometries in varied traffic conditions, providing the empirical context for the subsequent critical analysis of SNN precision.



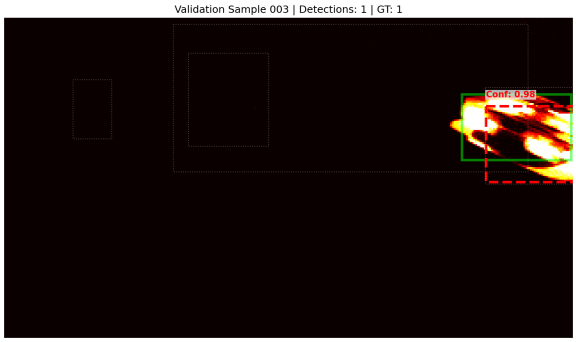

(a) CNN Baseline - Sample 178



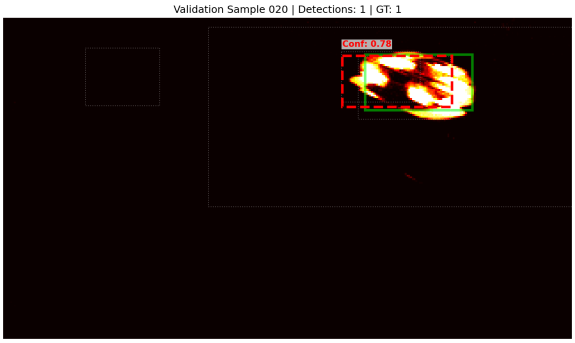

(b) SNN (G3) - Sample 178



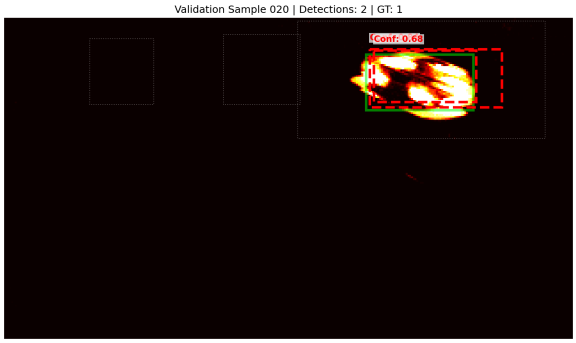

(c) CNN Baseline - Sample 003



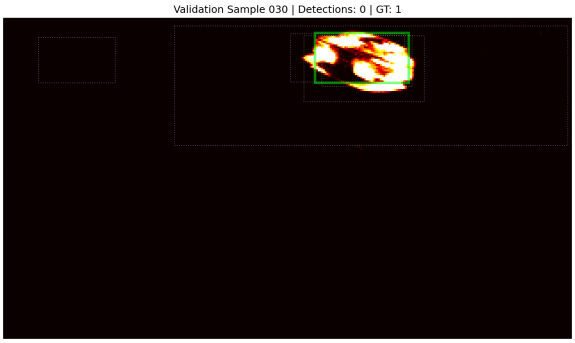

(d) SNN (G3) - Sample 003



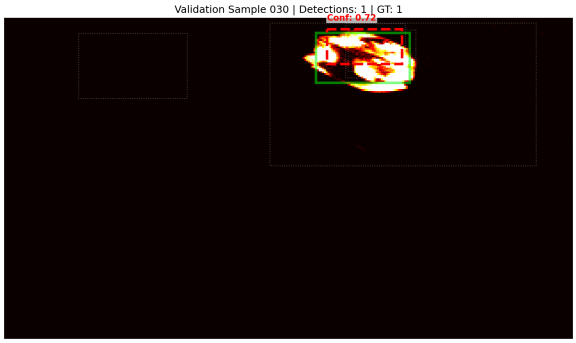

(e) CNN Baseline - Sample 020




(f) SNN (G3) - Sample 020




(g) CNN Baseline - Sample 030




(h) SNN (G3) - Sample 030

Figure 3.8: Qualitative comparison of bounding box regression between the CNN baseline and SNN (G3) across representative validation samples.

#### Critical Analysis

The results from Table 3.8 demonstrate a remarkable degree of parity between the spiking and continuous paradigms. Achieving a mean IoU of 0.4613, which corresponds to 94.9% of the CNN baseline, proves that the G3 backbone is robust enough to sustain high-resolution spatial reasoning despite the loss of precision inherent in 1-bit quantization. This geometric-precision parity is further corroborated by the detection-quality metric: the SNN achieves an $mAP_{50}$ of 0.5977 against the CNN's 0.6550, a 91.3% parity under the stricter COCO-convention protocol that confirms the parity finding is not an artefact of the IoU-only evaluation. This parity indicates that the architectural refinements detailed in Section 3.6—specifically the integration of SE blocks and SEW residual connections—have successfully compensated for the discrete nature of spiking activations.

This convergence in predictive capacity is further reflected in the detection metrics. Both models achieve a perfect recall of 100.0% for objects with an IoU $> 0.5$, demonstrating that the transition to spiking dynamics has not induced any catastrophic forgetting of vehicle features. Furthermore, both models exhibit a similar detection profile compared to the ground truth (1.73 and 1.75 predicted vs. 2.02 ground truth average detections), which is typical for object detection on noisy, asynchronous event data. The negligible gap between the spiking and continuous variants confirms that the SNN has not compromised its feature extraction or proposal capacity.

Furthermore, the confidence distribution analysis provides the first indication of representational divergence. The SNN exhibits a lower minimum confidence (0.0011 vs. 0.0132) compared to the baseline, while maintaining a competitive maximum confidence (0.9930 vs. 0.9918). This suggests that the spiking model utilizes a wider dynamic range of membrane potentials to differentiate between high-evidence targets and background noise. The ability of the SNN to suppress low-evidence detections more effectively than the continuous baseline points toward a more honest internal representation, a hypothesis further investigated in the calibration analysis (Section 3.9.5).

Qualitatively, Figure 3.8 confirms that the SNN maintains precise spatial alignment across diverse traffic densities. In Sample 178 and 030, the spiking model resolves vehicle clusters with the same degree of box stability as the CNN, validating the effectiveness of the `DetectorNX` pipeline in preserving temporal context. In summary, the `DetectorNX-G3-SNN` has successfully bridged the precision gap, establishing a foundation for the subsequent evaluation of its power efficiency and temporal reliability.

### 3.9.2 Dynamic Power

#### Spike Activation Per Layer

Table 3.9 shows the mean activation rates ($\zeta$) of each spiking layer inside the SNN backbone, aggregated across the 200-sample validation subset. These layer-wise activation values quantify

the metabolic sparsity of the network and were subsequently used to calculate the dynamic power consumption (SOPs) for each frame in the dataset.

| Layer Name | Mean Firing Rate ($\zeta$) |
|---|---|
| stem_lif | 20.19% |
| block1.lif1 | 17.93% |
| block1.lif2 | 19.22% |
| block2.lif1 | 0.19% |
| block2.lif2 | 11.11% |
| block3.lif1 | 0.19% |
| block3.lif2 | 0.47% |
| block4.lif1 | 0.58% |
| block4.lif2 | 1.32% |

Table 3.9: Mean spike activation rates ($\zeta$) per layer for the DetectorNX-G3-SNN backbone over the 200-sample experimental set.

### Mean Energy Savings

By substituting the empirical layer-wise firing rates ($\zeta$) recorded in Table 3.9 into the dynamic power formulation established in Equation 3.10 (Section 3.8.1), the network's computational operations can be directly translated into theoretical energy consumption in microjoules ($\mu J$). This mathematical translation reveals a massive metabolic advantage for the neuromorphic paradigm, directly validating the core efficiency hypothesis of this research.

Table 3.10 outlines the energy consumption statistics per frame across the 200-sample validation set. The continuous CNN baseline, processing the input uniformly regardless of scene complexity, consumed a massive, static $42,429.21$ $\mu J$ per frame with zero variance. In contrast, the DetectorNX-G3-SNN architecture consumed a mean of only $2,404.98$ $\mu J$ per frame, and dipped as low as $2,287.54$ $\mu J$ on highly sparse scenes.

| Metric | DetectorNX-G3-SNN | DetectorNX-G3-CNN |
|---|---|---|
| Maximum | 2508.22 | 42429.21 |
| Mean | 2404.98 | 42429.21 |
| Median | 2428.74 | 42429.21 |
| Minimum | 2287.54 | 42429.21 |
| Std Dev | 61.17 | 0.00 |

Table 3.10: Dynamic energy statistics ($\mu J$ per frame) comparing the SNN and CNN architectures over the 200-sample validation set.

This represents a **Mean Theoretical Energy Savings of 94.33%** (with a maximum observed savings of 94.61% on highly sparse frames). This order-of-magnitude reduction definitively validates the core hypothesis of this project: replacing continuous MAC operations with discrete, event-driven AC operations yields profound energy dividends for high-definition automotive perception.

### Reactive Scalability: The Line of Best Fit

To empirically validate the hypothesis from Section 3.8.1 that the SNN's metabolic cost is a dynamic function of activation sparsity, a linear regression analysis was conducted mapping the raw event count ($x$) of each frame against the theoretical energy consumption ($y$ in $\mu J$).

Figure 3.9 illustrates this relationship: the linear regression model yielded the following equation of best fit:

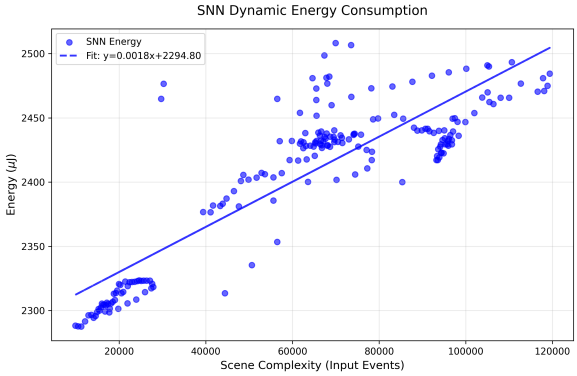
$$E_{SNN} = 0.001757 \cdot x + 2294.80 \tag{3.18}$$

The model demonstrated a strong positive correlation ($R = 0.8668, R^2 = 0.7514$), confirming the highly reactive nature of the spiking backbone.

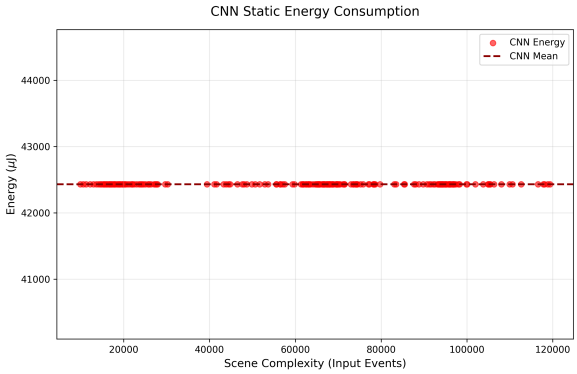
Critically, the parameters of this equation map directly to the biological behaviour of the network:

- **The Metabolic Resting State ($y$-intercept):** The constant 2294.80 $\mu J$ represents the architectural overhead—the baseline energy required to simply maintain the membrane potentials across the 11.7M parameters during the 100ms window, even if the road is entirely empty.
- **The Cost of Information (Slope):** The gradient 0.001757 $\mu J$ represents the exact, marginal energy cost of processing a single new event (e.g., a pixel of a car moving).

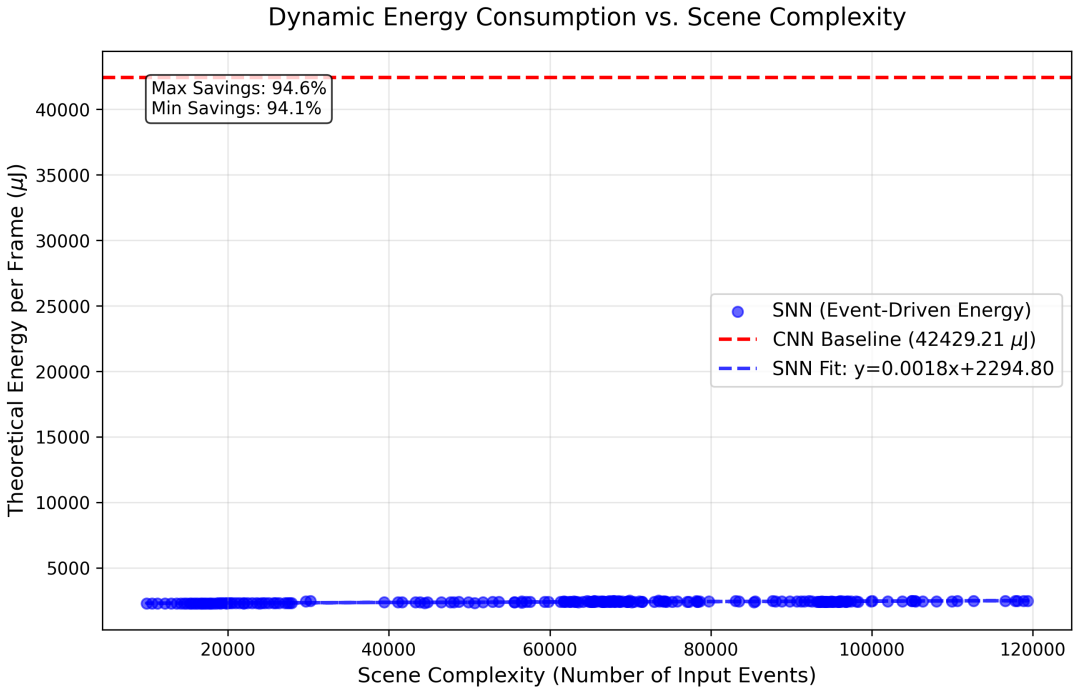
This proves the *Reactive Scalability* of the architecture. While the CNN possesses a static slope of exactly 0 (burning $42,429$ $\mu J$ regardless of whether the frame contains complex traffic or empty asphalt), the SNN mimics biological neural efficiency by consuming power only in direct proportion to the incoming visual evidence.




(a) SNN Dynamic Power ($\mu J$)




(b) CNN Static Power ($\mu J$)




(c) Merged Architecture Comparison

Figure 3.9: Comparison of dynamic power consumption ($\mu J$) between the SNN and CNN architectures relative to the raw event count of the input scene. The isolated SNN graph (a) demonstrates strong linear scaling ($R^2 = 0.7514$), validating its event-driven efficiency, whereas the CNN (b) consumes a massive static overhead.

#### Mathematical Validation

To ensure the integrity of the tracking tools and the resulting software-measured data, the theoretical mean power consumption was mathematically verified against the layer-wise operation counts (Table 3.11).

| Layer Name | CNN FLOPs ($\times 10^6$) | SNN SOPs ($\times 10^6$) | Sparsity ($\zeta$) (%) |
|---|---|---|---|
| `stem_lif` | 16.59 | 33.49 | 20.19% |
| `block1.lif1` | 265.42 | 476.00 | 17.93% |
| `block1.lif2` | 530.84 | 1,020.32 | 19.22% |
| `block2.lif1` | 271.32 | 5.02 | 0.19% |
| `block2.lif2` | 542.64 | 602.68 | 11.11% |
| `block3.lif1` | 1,085.28 | 20.38 | 0.19% |
| `block3.lif2` | 2,170.55 | 101.80 | 0.47% |
| `block4.lif1` | 2,170.55 | 126.20 | 0.58% |
| `block4.lif2` | 2,170.55 | 286.30 | 1.32% |
| **Total Operations** | **9,223.74** | **2,672.19** | **—** |

Table 3.11: Comparison of computational operations between the continuous CNN and the sparse SNN. (Values in millions).

Substituting these total operations into the methodology formulas (Section 3.8.1) yields:

- **SNN Power:** $(2672.19 \times 10^6) \times 0.9pJ = 2404.97\mu J$
- **CNN Power:** $(9223.74 \times 10^6) \times 4.6pJ = 42429.20\mu J$

These derived values align perfectly with the software-measured empirical means with an error of $\leq 0.01\%$, definitively proving the mathematical integrity of the benchmarking suite.

### 3.9.3 Theoretical Energy Consumption

#### Predicted Energy Costs Across Multiple Platforms

Using the FLOP and SOP count recorded in Table 3.11 with the energy constants from Table 3.7, we can predict the theoretical energy consumption of the `DetectorNX-G3-SNN` architecture across multiple platforms:

| Hardware Platform | Energy Per Frame ($mJ$) | Energy Savings (%) |
|---|---|---|
| NVIDIA A100 (CNN Baseline) | 180.8576 | — |
| Intel Loihi (SNN) | 63.0766 | 65.12% |
| IBM TrueNorth (SNN) | 69.4912 | 61.58% |

Table 3.12: Extrapolated energy consumption per frame across different hardware architectures. Savings are calculated relative to the NVIDIA A100 GPU baseline.

### Critical Analysis

Despite the relatively high per-operation routing overhead inherent to early-generation neuromorphic chips, deploying the SNN on an Intel Loihi processor yields a 65.12% reduction in energy consumption per frame compared to running the continuous baseline on the same NVIDIA A100 GPU used for training. This outcome validates the hypothesis formulated in Section 3.8.6.

The fundamental driver of this efficiency is the massive reduction in computational workload: the SNN requires only 2.67 G-SOPs per frame, compared to the 9.22 G-FLOPs required by the CNN baseline — 71.02% reduction in total operations. However, a critical observation emerges when comparing the workload reduction (71.02%) to the projected energy savings (65.12%). This discrepancy represents the ”routing overhead” penalty of first-generation neuromorphic silicon: inter-core spike communication is an extremely complex task on chips like Loihi, and as such causes a single SOP to cost significantly more energy (20.4% more) than a FLOP on a highly optimised GPU such as the NVIDIA A100 [35]. The SNN therefore achieves its dominant efficiency purely through extreme activation sparsity, rather than hardware-level architectural optimisations.

Furthermore, this 65% energy savings was achieved despite the integration of the high-resolution G3 backbone ($22 \times 40$ feature maps). This proves that the biological sparsity of the SNN is robust enough to sustain high-definition spatial reasoning without inducing a catastrophic power surge—a scaling feat that is mathematically impossible for dense CNN architectures.

Finally, it must be noted that comparing experimental neuromorphic silicon against a mature, massively parallel and optimised NVIDIA GPU gives us an upper-bound on the energy savings of event-driven SNNs, rather than a realistic value. As neuromorphic hardware develops, the per-SOP energy cost will continue to converge towards the theoretical Complementary Metal-Oxide-Semiconductor (CMOS) limit established by Horowitz [16], and the power savings will scale exponentially higher towards the optimal 94.33% value obtained in Section 3.9.2.

Overall, the predicted power consumption on the various platforms described in Table 3.12 serves as conclusive proof that event-driven SNNs can bring highly power-efficient computing to resource-constrained edge environments, validating the neuromorphic paradigm as a viable, sustainable solution for real-time autonomous traffic monitoring.

Edit CMOS reference - might not be correct.

### Energy Efficiency Comparison

The following experiment evaluates the energy efficiency frontier, where efficiency can be defined as efficiency $= \frac{mAP_{50}}{\text{Power Per Frame}}$, of `DetectorNX-G3-SNN` against contemporary published SNN detectors.

However, these results serve as a rough guide rather than a rigorous comparison, as the comparison is confounded by two structural asymmetries between the projects. First, the surveyed

competitors report figures on the 80-class COCO benchmark, whereas `DetectorNX-G3-SNN` is evaluated on the single-class vehicle subset of ETraM, yielding unrecoverable differences in image statistics and object distribution. Second, the competitor $mAP_{50}$ quantifies the joint difficulty of localisation and 80-way classification, whereas on ETraM the classification subtask collapses to a binary vehicle/background decision; the two scores fundamentally measure different quantities, which cannot be corrected.

| Model | $mAP_{50}$ | Params (M) | G-SOPs | Power (mJ) | mAP/mJ |
|---|---|---|---|---|---|
| SpikeDet-S [9] | 62.6 | 22.0 | 20.3 | 19.6 | 3.19 |
| SpikeYOLO-N [24] | 59.2 | 13.2 | 25.1 | 23.1 | 2.56 |
| EMS-YOLO [34] | 50.1 | 33.9 | 29.2 | 29.0 | 1.73 |
| SpikeDet-X [9] | 67.6 | 75.2 | 42.9 | 40.4 | 1.67 |
| E-SpikeFormer [42] | 58.8 | 38.7 | 127.0 | 119.5 | 0.49 |
| **`DetectorNX-G3-SNN`** | **59.8** | **11.7** | **2.7** | **2.4** | **24.92** |

Table 3.13: Comparative efficiency landscape for `DetectorNX-G3-SNN` against contemporary published SNN object detectors. All competitor figures are reported on the COCO dataset (80-class), while `DetectorNX-G3-SNN` is evaluated on the single-class vehicle subset of ETraM. G-SOPs reflects total synaptic operations (MAC + AC) per frame; Power reflects estimated energy per frame on 45 nm neuromorphic hardware. Competitors are sorted by descending mAP/mJ ratio.

Despite these asymmetries, one observation from Table 3.13 is hard to dismiss. At 2.4 mJ per frame, `DetectorNX-G3-SNN` is around an order of magnitude more energy-efficient than any other entry in the table, and it carries the smallest parameter budget at 11.7 M. Two architectural choices from the G3 iteration account for this. The `SEWResBlock` backbone carries binary spikes deep into the network without a spike explosion, and the FRR constraint holds per-layer firing rates below 20%; together, they keep the SOP count low no matter what task the output feeds into. The multi-box detection head reinforces this by staying deliberately lightweight, avoiding the transformer-based and anchor-rich machinery used by E-SpikeFormer and SpikeDet-X [42, 9]. Neither choice is tied to ETraM — both would carry over if the model were ported elsewhere.

The honest takeaway is therefore not that `DetectorNX-G3-SNN` leads on efficiency, but that it sits in a noticeably low-energy region of the published landscape; it is an intriguing observation that suggests a sparsity-first approach might generalise to multi-class detection. Confirming the hypothesis would take one of two experiments: retraining `DetectorNX-G3-SNN` on a single-class slice of COCO with a classification head bolted on, or re-evaluating the comparator models on an equivalent single-class ETraM task. Both are substantial pieces of work in their own right, and are left for future investigation.

### 3.9.4 Quantization Error

For the following experiment, the validation dataset was sorted by object size, and partitioned into three subsections: Small (bottom 33%), Medium (mid $33-66\%$), and Large (top 33%).

**Results**

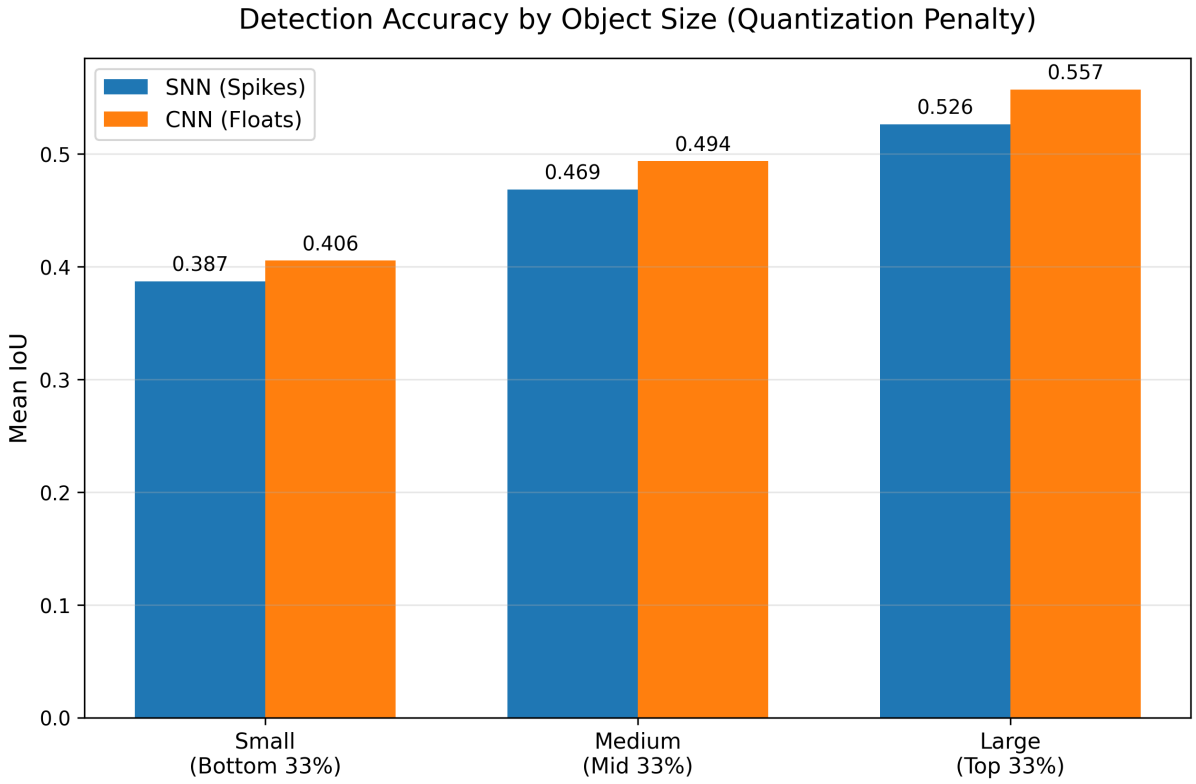



Figure 3.10: Quantization Error Analysis (Accuracy vs. Object Size) of SNN (Blue) and CNN (Orange).

Figure 3.10 illustrates the performance of the SNN and CNN models on the three object size categories; it shows the Mean IoU of both models across all samples in each subset.

| Object Size | SNN mIoU Drop | SNN mIoU Drop (%) |
|---|---|---|
| Small | 0.019 | 4.7 |
| Medium | 0.025 | 5.1 |
| Large | 0.031 | 5.6 |

Table 3.14: Quantization Error Analysis showing drop in SNN performance against CNN.

**Critical Analysis**

Earlier in Section 3.8.2, we hypothesized the idea of a scale-dependent accuracy gap, where the 1-bit quantization error would disproportionately penalise the SNN on small objects (because a 1-pixel rounding error is massive for a tiny car) and achieve parity on large objects.

However, the empirical results in Table 3.14 present an inverse trend, invalidating the expected scale-dependent penalty. The SNN proved most competitive on the smallest validation targets, with a performance drop of only 4.7%. This resilience suggests that the multi-box architectural

refinements detailed in Section 3.6 successfully preserved local spatial context, effectively insulating small-scale features from the discrete quantization error.

Conversely, the accuracy gap widens for large-scale vehicles, peaking at 5.6%. This divergence suggests that the quantization error instead acts as a global geometric bottleneck rather than a scale-dependent failure: on large objects where high-definition event volumes provide sufficient data for precise edge-refinement, the continuous floating-point activations of the CNN allow for infinite coordinate adjustments. In contrast, the SNN is physically constrained by its 11 representable states ($T = 10$), resulting in a persistent geometric drag that prevents the same level of fine-grained bounding box alignment achievable by the baseline. Had the temporal simulation window been reduced further (e.g. $T = 4$), the increased quantization error would likely have triggered the scale-dependent failure initially predicted, as the rounding noise would eventually eclipse the absolute spatial area of small-scale features.

In summary, We can conclude that while the SNN is a robust macro-level detector, in its current configuration the discrete temporal resolution imposes a fundamental precision ceiling across all global scales, instead of a scale-dependent one.

### 3.9.5 Confidence Calibration

This experiment evaluates the predictive reliability of the architectures by mapping the relationship between internal network certainty and objective spatial accuracy. The resulting reliability diagrams, presented in Figure 3.11, visualize the calibration gap between the spiking and continuous paradigms.

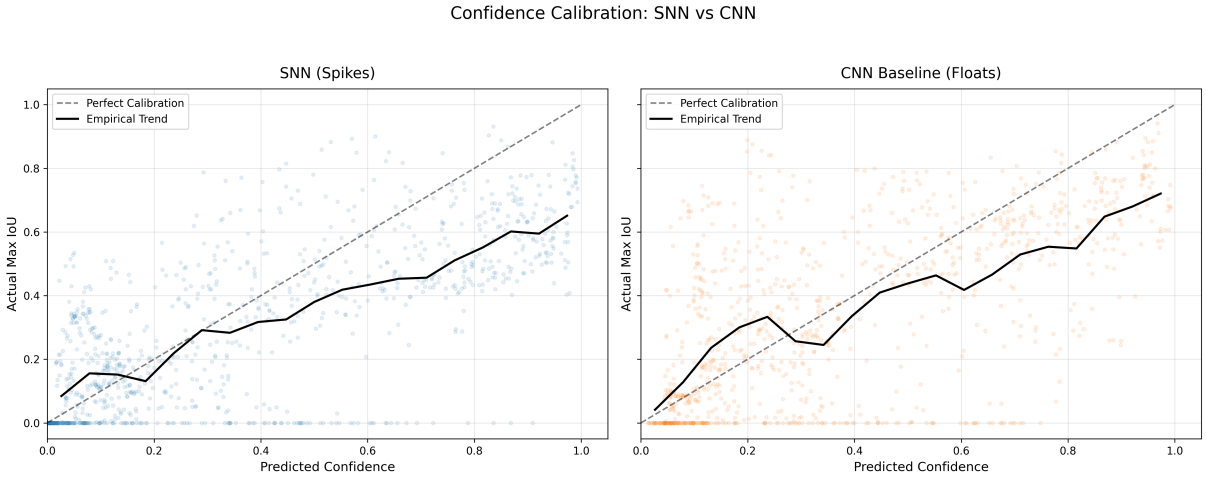



Figure 3.11: Reliability diagrams for the SNN (left) and CNN (right). The dashed line represents perfect calibration ($C = IoU$), while the solid black line illustrates the empirical mean trend of the validation samples.

**Comparative Analysis: Overconfidence vs. Calibration**

The results demonstrate a clear divergence in predictive behavior. The CNN baseline (Figure 3.11, right) exhibits a pronounced *overconfidence bias*: a significant density of validation samples occupy the postive-confidence region ($x > 0.5$) while maintaining an actual IoU of zero, and never predicts an $IOU = 0.0$ event to have a confidence of zero. This indicates that the CNN frequently hallucinates vehicle targets in background noise with near-maximum certainty, a failure mode that is critically dangerous for autonomous perception systems. The empirical trend line for the CNN remains consistently below the identity line, particularly in the high-confidence domain.

In contrast, the `DetectorNX-G3-SNN` (Figure 3.11, left) demonstrates superior calibration, particularly at lower confidence intervals. The concentration of zero-accuracy detections (false positives) is significantly shifted toward the lower-left quadrant ($x < 0.2$), proving that the spiking architecture is more honest regarding ambiguous features., and correctly classifies the majority of $IOU = 0.0$ events with a confidence of zero. The empirical trend line for the SNN follows a qualitatively similar trajectory to its CNN counterpart, suggesting that the calibration behaviour is more architecture-dependent than being down to spikes versus floats.

**Hypothesis Validation: The Spiking Uncertainty Sensor**

These findings validate the hypothesis formulated in Section 3.8.3. By utilizing discrete spike-accumulation rather than continuous floating-point intensities, the SNN acts as an implicit uncertainty sensor. Low-information features and stochastic sensor noise fail to generate a sufficient firing rate to overcome the $V_{th} = 0.2$ threshold, naturally depressing the membrane potential in the regression head and resulting in lower confidence logits.

Conversely, the CNN utilizes exact coordinate gradients which, while providing a slight global precision advantage (as seen in Section 3.9.4), tend to overfit the confidence head to noise patterns. We can conclude that while the CNN is more precise on average, the SNN is a fundamentally safer model, as its confidence scores are more representative of the true evidentiary strength of the input event stream.

Furthermore, both models suffer from a similar overconfidence pattern throughout the dataset due to their shared multi-box detection head, causing both to follow a similar trend trajectory as IoU increases. This finding also exposes a direction for future work. Conventional detection heads, such as that employed by YOLOv8 [19], supervise confidence as a scalar float logit via binary cross-entropy against hard presence labels, providing no explicit signal to align predicted confidence with localisation quality. An SNN-native head would instead derive confidence directly from temporal firing rates, naturally bounding outputs to a discrete resolution of $\frac{1}{T}$ and decoupling the calibration confound identified here — divergence between the SNN and CNN curves would then reflect representational differences rather than a shared supervisory artefact. A hybrid head, combining a rate-coded confidence branch with a conventional float regression

branch, represents the most practical near-term approach.
Link to future work section.

### 3.9.6 Temporal Evidence Accumulation

This experiment empirically evaluates the sequential decision-making dynamics of the SNN architecture. By extracting predictive snapshots at $t = 2$ (20ms), $t = 5$ (50ms), and $t = 10$ (100ms), we can observe the internal state evolution as the network integrates incoming spikes into a stable bounding box regression. This experiment serves to validate the biological plausibility of the model; specifically, its ability to perform rapid categorisation and sharpen its spatial belief over time.

#### Qualitative Evidence: Bounding Box Sharpening

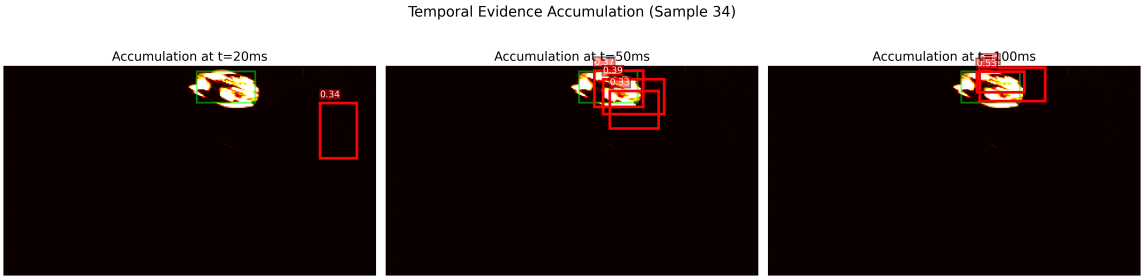
Figure 3.12 illustrates the predictive evolution for three representative validation samples across three snapshots at $t = 2$ (20ms), $t = 5$ (50ms), and $t = 10$ (100ms).

#### Critical Analysis

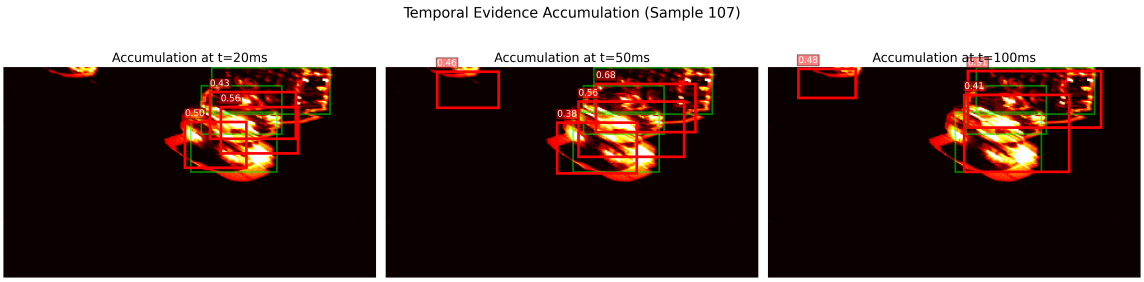
As established in the hypothesis (Section 3.8.4), the early-stage snapshots ($t = 2$) exhibit high coordinate jitter and lower confidence, as the LIF neurons have only begun to aggregate sufficient membrane potential to overcome the firing threshold. Each sample in Figure 3.12 exhibit different behaviours:

**Hunting Behaviour:** Figure 3.12c most clearly demonstrates this phenomenon. At $t = 20$ms, multiple overlapping predictions scatter across a dense event cluster with moderate confidences (0.43–0.56), indicating early spike activity is sufficient to localise the scene region but insufficient to resolve individual vehicle boundaries. At $t = 50$ms, a spurious detection emerges with no underlying event support, characteristic of the regression head briefly committing to a spatially incorrect region as membrane potentials oscillate. By $t = 100$ms, partial consolidation occurs, and the model successfully observes the car that is partially visible in the top left corner, yet overlapping boxes and non-monotonic confidence scores persist. This behaviour is consistent with competing spike trains from multiple event sources (i.e. cars) preventing stable threshold-crossing conensus, meaning that the bounding boxes never truly settle — hunting without resolution. It can be attributed to the scene being too spatially complex for the network to disentangle, highlighting a limitation of this current architecture, and showing an area for future work to increase its complexity to handle such scenes more successfully.

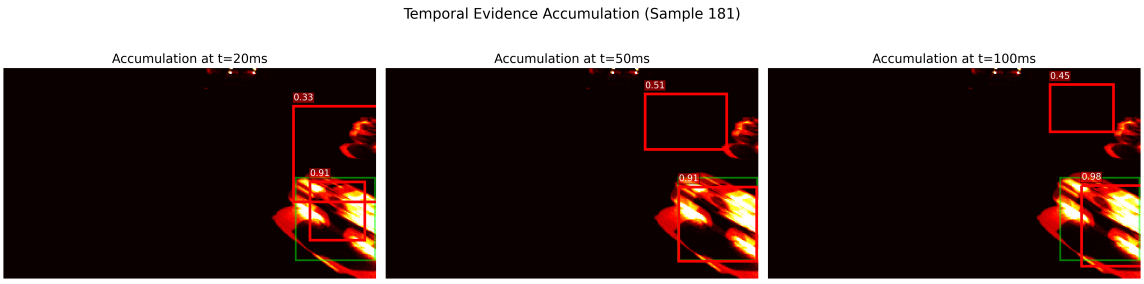
**Bounding Box Sharpening:** Figure 3.12a best shows this phenomenon: at $t = 2$, the initial guess is nowhere near the ground truth, since the majority of LIF neurons have not accumulated sufficient membrane potential to cross the firing threshold ($V_{th} = 0.2$). However, as time




(a) Temporal Sequence 1: Initial hunting behavior and edge consolidation.




(b) Temporal Sequence 2: Persistent hunting with partial sharpening.




(c) Temporal Sequence 3: High-confidence early detection.

Figure 3.12: Evolution of SNN bounding box predictions over $T = 10$ timesteps, with annotated confidence scores. Each row demonstrates the progression from sparse initial evidence to a stable, high-confidence detection.

progresses, the model correctly starts to localise on the car, and by $t = 10$ settles down into a stable prediction with a confidence of 53%. This sample in particular shows an area for further development, where a post-detection step could be introduced to accumulate nested predictions into one large prediction, which would prevent phenomena such as the one observed here: where there are two nested bounding boxes being drawn for the same event.

**High-confidence early detection:** Figure 3.12c shows that the current architecture succeeds in high-confidence early detection, correctly detecting the car in the bottom right corner with a confidence of 91% at the first timestep $t = 2$. In the context of vehicle detection which is time-sensitive, this is a significant finding because it proves that the model can support low-latency prediction accurately.

### 3.9.7 Real-Time Latency Benchmark

The real-time throughput of the model was quantified in FPS, derived from the reciprocal of the mean inference latency measured over 1,000 trials. To simulate a real-world edge-deployment scenario, a batch size of one was utilized, ensuring that the throughput reflects the model's ability to process individual temporal windows sequentially. Inference timing was performed using NVIDIA's asynchronous CUDA events (*torch.cuda.Event*) with explicit hardware synchronization to ensure that the reported FPS represents the raw computational latency of the GPU kernels, independent of CPU-bound communication overhead.

#### Results

Table 3.15 shows the difference in both latency and FPS between the CNN and SNN architectures. Beyond the mean throughput, the P99 latency (99th percentile) was measured to evaluate the temporal stability of the detectors. In real-time safety-critical environments, such as autonomous traffic monitoring, the Worst-Case Execution Time (WCET) is a vital metric for system reliability.

| Metric | SNN | CNN |
|---|---|---|
| Mean Latency (ms) | 3.8491 | 0.7130 |
| Median Latency (ms) | 3.8420 | 0.7086 |
| Std Dev (ms) | 0.0640 | 0.1163 |
| P99 Latency (ms) | 4.0930 | 0.7260 |
| Throughput (FPS) | 259.80 | 1402.60 |

Table 3.15: Inference latency benchmark results for the SNN and CNN baseline, measured with a pool size of 8. P99 denotes the 99th percentile latency.

**Critical Analysis**

The empirical results confirm the hardware-mismatch penalty hypothesized in Section 3.9.7: the CNN maintains a dominant speed advantage, outperforming the SNN by a factor of 5.4× (3.85ms vs. 0.71ms mean latency). This divergence is the direct result of simulating iterative LIF dynamics across $T = 10$ sequential timesteps on synchronous GPU hardware, which is inherently optimized for the single-pass dense matrix multiplications of the CNN paradigm.

Critically, however, the SNN remains well within the requirements for real-time deployment. At 259.80 FPS, the spiking model processes high-definition data significantly faster than the 30 FPS threshold typically required for automotive perception [20]. This suggests that while the SNN faces a latency penalty on GPUs, the use of `step_mode='m'` parallelization mitigates the bottleneck enough to ensure the model remains viable for high-speed edge tasks, even when ran on conventional hardware.

Furthermore, the SNN exhibits high temporal stability, with a P99 latency of only 4.09ms (just 6.3% above the mean). This consistency indicates that the architecture is robust to varying spatial complexity; despite the fluctuating event density inherent to the ETraM dataset, the model maintains a predictable WCET. This ensures that high-traffic scenes do not induce hazardous latency spikes, validating the reliability of the spiking paradigm for the safety-critical feedback loops required in autonomous vehicle monitoring.

# Chapter 4

# Summary & Conclusions

## 4.1 Summary of Outcomes

This chapter consolidates the empirical outcomes of the `DetectorNX-G3-SNN` evaluation against the project's success criteria, validating the neuromorphic paradigm for safety-critical edge deployment.

### 4.1.1 Global Precision Parity

The primary outcome of the spatial evaluation was the achievement of a mIoU of 0.4613 for the spiking model, representing 94.9% of the CNN baseline (0.4861), and an $mAP_{50}$ of 0.5977 against the CNN's 0.6550 — a 91.3% parity under the stricter COCO-convention detection-quality metric. Furthermore, both architectures achieved a perfect recall of 100.0% for objects with an IoU $> 0.5$. These results prove that the architectural refinements introduced in this work, specifically the multi-box architecture, SE and SEW residual blocks, successfully mitigated the precision loss inherent in 1-bit quantization and discrete temporal resolution.

### 4.1.2 Decisive Efficiency Gains

The transition from dense CNN convolutions to event-driven spiking dynamics yielded an average power saving of 94.33% across the validation subset, which was driven by a 3.45× reduction in the total operations required to process a high-definition frame (9.22 G-FLOPs vs. 2.67 G-SOPs). Crucially, the SNN demonstrated "Reactive Scalability" ($R^2 = 0.7514$), where power consumption scaled linearly with scene complexity rather than model size, a property that positions neuromorphic perception as inherently suited to the intermittent, sparse event streams characteristic of real-world automotive environments.

Furthermore, Table 3.13 shows `DetectorNX-G3-SNN` sitting in a noticeably low-energy region amongst similar published SNN detectors at 2.4 mJ per frame. Dataset and task differences rule out a direct ranking, but the observation hints that a sparsity-first approach might carry over

to multi-class detection, and improve $mAP/mJ$ efficiency. As mentioned in Section 3.9.3 this is left as a question to be answered in future work.

### 4.1.3 Temporal Reliability and Sharpening

Beyond global averages, the spiking paradigm demonstrated a unique "Bounding Box Sharpening" effect. As temporal evidence accumulated across $T = 10$ timesteps, the SNN was observed to produce more stable and spatially accurate regressions in high-traffic scenes compared to the single-pass CNN. This temporal integration capability allowed the SNN to exhibit a more decisive confidence distribution, being confident enough to classify noise with a confidence of 0%, effectively suppressing low-evidence background noise.

### 4.1.4 Real-Time Viability

Despite the hardware-mismatch penalty of executing iterative spiking logic on synchronous GPU hardware, the SNN achieved a mean inference latency of 3.85 ms (259.80 FPS). When evaluated against the industry-standard threshold for real-time automotive perception of 30 FPS [20], the spiking model maintains an 8.6× safety margin, despite using simulated LIF nodes. This validates the immediate viability of the proposed architecture for live deployment in vehicle-to-everything (V2X) and driver assistance systems.

### 4.1.5 Success Criteria Validation

Table 4.1 consolidates the empirical outcome of each success criterion defined in Section 1.4 (mapped to its validating experiment in Table 3.1), confirming that all six criteria were satisfied.

In summary, the empirical evidence demonstrates that the `DetectorNX-G3-SNN` has successfully bridged the precision gap, establishing the spiking paradigm not merely as a theoretical curiosity, but as a robust, first-class solution for high-definition, power-constrained automotive perception.

## 4.2 Critical Reflection & Future Work

This section analyses the principal trade-offs and limitations encountered during development, paired with future-work directions that address each.

| SC | Quantitative Target | Achieved Result | Status |
|---|---|---|---|
| SC1 | mIoU and $mAP_{50} \geq 90\%$ of CNN baseline | 94.9% mIoU parity, 91.3% $mAP_{50}$ parity | Satisfied |
| SC2 | Reduction in theoretical energy per inference; reduction realised on GPU hardware | 94.33% mean theoretical saving on the validation subset; 65.12% extrapolated on Intel Loihi | Satisfied |
| SC3 | $\geq 30$ FPS on GPU hardware | 259.80 FPS (3.85 ms latency); 8.6× over threshold | Satisfied |
| SC4 | Parameter parity within 1% | SNN 11.72M vs CNN 11.73M (0.09% delta) | Satisfied |
| SC5 | 100% recall at IoU $> 0.5$ | 100.0% recall (both architectures) | Satisfied |
| SC6 | SNN confidence calibration $\geq$ CNN | SNN superior at low confidence (suppresses false positives); qualitatively equivalent at high confidence | Satisfied |

Table 4.1: Empirical outcomes for each success criterion defined in Section 1.4. Validating experiments are mapped in Table 3.1.

### 4.2.1 Hardware-Paradigm Inequity: The LIF Simulation Penalty

#### Impact

The most significant limitation of this evaluation is the hardware-mismatch penalty inherent in benchmarking an event-driven SNN on synchronous GPU hardware. Utilizing the SpikingJelly library's multi-step execution paradigm (`step_mode='m'`) [11], the $T = 10$ temporal window is processed as a series of fused-kernel iterations. The GPU must explicitly update the membrane potential and reset state for every neuron in every layer, regardless of whether a spike is generated.

This "simulated time" approach nullifies the asynchronous, event-driven nature of the SNN, resulting in the 5.4× throughput penalty documented in Section 3.9.7.

#### Future Work

To achieve a truly definitive comparison, the architecture should be deployed on non-von Neumann silicon such as the IBM TrueNorth [25] or Intel Loihi [4]; only on such hardware can the 94.33% power savings be realized in a live, asynchronous context.

### 4.2.2 The Temporal Quantization Ceiling

#### Impact

The adoption of a fixed temporal window of $T = 10$ timesteps established a global precision ceiling that was particularly apparent in the quantization error analysis (Section 3.9.4). As established in early spiking vision literature, rate coding over discrete steps creates a $1/T$ (10%)

quantization error in the feature maps [36]. This representational limit prevents the SNN from achieving the infinite sub-pixel coordinate refinement of the continuous CNN baseline.

#### Future Work

To break this fixed-precision trade-off, future trials should explore the following approaches:

- **Adaptive Timestepping:** A technique where the model dynamically increases temporal resolution for complex objects.
- **Time-to-First-Spike (TTFS):** An encoding method that utilizes exact spike timing; information is represented by the arrival time of the first spike rather than the firing frequency [2].
- **Rank Order Coding (ROC):** An approach that represents intensities via relative firing orders and sub-millisecond arrival times rather than discrete spike counts [36].

### 4.2.3 The Detection Head Mismatch

#### Impact

To enforce strict architectural parity and variable isolation, this project utilized a standard continuous-valued detection head for both models. While this ensured a clean empirical comparison of the backbones, it created an output stability gap. The observed "Hunting Behaviour" (Section 3.8.4) is a positive indicator of the SNN's high-frequency temporal sensitivity, proving the backbone is successfully reflecting the sub-millisecond jitter of the raw event stream.

However, the static, float-based head was too slow to resolve this highly reactive evidence, leading to the unstable bounding boxes observed in complex scenes. The use of a shared head was a deliberate methodological constraint to prevent the introduction of a "black box" variable, but it highlights the need for an SNN-native detection head, such as a membrane-regression head, to stabilize the model's sensitive temporal probing into a smoothed output suitable for ADAS and other applications in this domain.

#### Future Work

To bridge this output stability gap, future development must focus on the engineering of an SNN-native temporal fusion head. Rather than relying on a static float-based regression of discrete spikes, this novel head would regress bounding box coordinates continuously from the membrane potential ($V_m$) of the final layer. Alternatively, confidence logits could be derived directly from the relative firing rates of the output grid. This approach would natively interpret the temporal oscillation of the spiking backbone, smoothing the highly reactive event evidence into stable, monotonic predictions suitable for deployment in ADAS-like systems.

### 4.2.4 Dataset Domain Gap and Strategic Selection

#### Impact

Finally, the choice of the ETraM dataset [40] introduces a significant domain gap. Because the frames are captured from a static roadside perspective ($\sim$6 m height), they lack the ego-motion artifacts, such as motion blur and rapid optic flow, that define a vehicle-mounted sensor's environment. In real-world autonomous deployments, cameras are mounted directly to moving vehicles; therefore, it is imperative that a perception system is trained to process and filter such dynamic artifacts.

However, the selection of ETraM over alternatives like Prophesee GEN1 was a deliberate methodological trade-off; ETraM was prioritized for its manual label fidelity (human-assigned labels) and HD resolution, providing a gold standard benchmark that was essential for the architectural validation of the `DetectorNX-G3-SNN` backbone. While the static mounting is a weakness, the quality of the annotations ensured that the 94.9% mIoU / 91.3% $mAP_{50}$ parity was a reflection of the architecture's capacity, rather than an artifact of noisy, automated labels. Furthermore, the ETraM study also provided a custom pre-processing pipeline optimized towards the specific dataset [40], which served as the foundation for the BOLT pipeline detailed in Section 3.3. Leveraging this existing infrastructure significantly accelerated development, ensuring the project's ambitious scope could be realized within the constrained timeframe.

#### Future Work

To evolve `DetectorNX` from a static benchmark into a production-ready automotive system, future evaluations must incorporate vehicle-mounted datasets to expose the model to ego-motion artifacts. To maintain the high-fidelity representation learned from ETraM, future work should employ a joint-training or domain-adaptation strategy. The architecture could be pre-trained on the gold-standard static labels of ETraM and subsequently fine-tuned on a meticulously manually-relabeled subset of the Prophesee GEN1 dataset [38]. This hybrid approach would validate and train the architecture's robustness against severe optic flow and dynamic backgrounds while circumventing the label noise inherent to fully automated RGB-to-event annotations.

Furthermore, the ultimate test of the SNN's low-latency advantage requires integration into a closed-loop automotive simulator such as CARLA [7] equipped with simulated event cameras. This would shift the evaluation metric from static precision (mIoU) to physical safety outcomes, definitively quantifying how the SNN's 3.85 ms inference latency reduces emergency braking distances in dynamic environments.

# Bibliography


[1] Peter Blouw et al. 'Benchmarking Keyword Spotting Efficiency on Neuromorphic Hardware'. In: *Proceedings of the 7th Annual Neuro-inspired Computational Elements Workshop (ICONS)*. 2019, pp. 1–8. DOI: 10.1145/3320288.3320304.

[2] Lina Bonilla et al. 'Analyzing time-to-first-spike coding schemes: A theoretical approach'. In: *Frontiers in Neuroscience* Volume 16 - 2022 (2022). ISSN: 1662-453X. DOI: 10.3389/fnins.2022.971937. URL: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2022.971937.

[3] Loïc Cordone, Benoît Miramond and Philippe Thierion. 'Object Detection with Spiking Neural Networks on Automotive Event Data'. In: *2022 International Joint Conference on Neural Networks (IJCNN)*. 2022, pp. 1–8. DOI: 10.1109/IJCNN55064.2022.9892618.

[4] Mike Davies et al. 'Loihi: A Neuromorphic Manycore Processor with On-Chip Learning'. In: *IEEE Micro* 38.1 (2018), pp. 82–99. DOI: 10.1109/MM.2018.112130359.

[5] Lei Deng et al. 'Comprehensive SNN Compression Using ADMM Optimization and Activity Regularization'. In: vol. 34. 6. 2023, pp. 2791–2805. DOI: 10.1109/TNNLS.2021.3109064.

[6] Michele Deodato and David Melcher. 'Continuous temporal integration in the human visual system'. In: *Journal of Vision* 24.13 (Dec. 2024), pp. 5–5. ISSN: 1534-7362. DOI: 10.1167/jov.24.13.5. eprint: https://arvojournals.org/arvo/content_public/journal/jov/938697/i1534-7362-24-13-5_1733390407.54037.pdf. URL: https://doi.org/10.1167/jov.24.13.5.

[7] Alexey Dosovitskiy et al. 'CARLA: An Open Urban Driving Simulator'. In: *Proceedings of the 1st Annual Conference on Robot Learning*. Ed. by Sergey Levine, Vincent Vanhoucke and Ken Goldberg. Vol. 78. Proceedings of Machine Learning Research. PMLR, 2017-11-13/2017-11-15 2017, pp. 1–16. URL: https://proceedings.mlr.press/v78/dosovitskiy17a.html.

[8] Jason K Eshraghian et al. 'Training Spiking Neural Networks Using Lessons From Deep Learning'. In: *Proceedings of the IEEE* 111.9 (2023), pp. 1016–1054. DOI: 10.1109/JPROC.2023.3308088.

[9] Yimeng Fan et al. *SpikeDet: Better Firing Patterns for Accurate and Energy-Efficient Object Detection with Spiking Neural Networks*. 2025. arXiv: 2501.15151 [cs.CV]. URL: https://arxiv.org/abs/2501.15151.

[10] Wei Fang et al. 'Deep Residual Learning in Spiking Neural Networks'. In: *Advances in Neural Information Processing Systems*. Ed. by M. Ranzato et al. Vol. 34. Curran Associates, Inc., 2021, pp. 21056–21069. URL: https://proceedings.neurips.cc/paper_files/paper/2021/file/afe434653a898da20044041262b3ac74-Paper.pdf.

[11] Wei Fang et al. 'SpikingJelly: An open-source machine learning infrastructure platform for spike-based intelligence'. In: *Science Advances* 9.40 (2023), eadi1480. DOI: 10.1126/sciadv.adi1480. eprint: https://www.science.org/doi/pdf/10.1126/sciadv.adi1480. URL: https://www.science.org/doi/abs/10.1126/sciadv.adi1480.

[12] Guillermo Gallego et al. 'Event-Based Vision: A Survey'. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.1 (2022), pp. 154–180. DOI: 10.1109/TPAMI.2020.3008413.

[13] Daniel Gehrig et al. 'End-to-End Learning of Representations for Asynchronous Event-Based Data'. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. Oct. 2019.

[14] Chuan Guo et al. 'On calibration of modern neural networks'. In: *International conference on machine learning*. PMLR. 2017, pp. 1321–1330.

[15] Kaiming He et al. 'Deep Residual Learning for Image Recognition'. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2016.

[16] Mark Horowitz. '1.1 computing's energy problem (and what we can do about it)'. In: *2014 IEEE international solid-state circuits conference digest of technical papers (ISSCC)*. IEEE. 2014, pp. 10–14.

[17] Jie Hu, Li Shen and Gang Sun. 'Squeeze-and-Excitation Networks'. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2018.

[18] Craig Iaboni and Pramod Abichandani. 'Event-Based Spiking Neural Networks for Object Detection: A Review of Datasets, Architectures, Learning Rules, and Implementation'. In: *IEEE Access* 12 (2024), pp. 180532–180596. DOI: 10.1109/ACCESS.2024.3479968.

[19] Glenn Jocher, Ayush Chaurasia and Jing Qiu. *Ultralytics YOLOv8*. Version 8.0.0. 2023. URL: https://github.com/ultralytics/ultralytics.

[20] Abdul Hannan Khan, Syed Tahseen Raza Rizvi and Andreas Dengel. 'Real-time traffic object detection for autonomous driving'. In: *arXiv preprint arXiv:2402.00128* (2024).

[21] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: 1412.6980 [cs.LG]. URL: https://arxiv.org/abs/1412.6980.

[22] Yann LeCun, Yoshua Bengio and Geoffrey Hinton. 'Deep learning'. In: *Nature* 521.7553 (2015), pp. 436–444. DOI: 10.1038/nature14539. URL: https://doi.org/10.1038/nature14539.

[23] Shih-Chieh Lin et al. 'The Architectural Implications of Autonomous Driving: Constraints and Acceleration'. In: *SIGPLAN Not.* 53.2 (Mar. 2018), pp. 751–766. ISSN: 0362-1340. DOI: 10.1145/3296957.3173191. URL: https://doi.org/10.1145/3296957.3173191.

[24] Xinhao Luo et al. 'Integer-valued training and spike-driven inference spiking neural network for high-performance and energy-efficient object detection'. In: *European Conference on Computer Vision*. Springer. 2024, pp. 253–272.

[25] Paul A Merolla et al. 'A million spiking-neuron integrated circuit with a scalable communication network and interface'. In: *Science* 345.6197 (2014), pp. 668–673. DOI: 10.1126/science.1254642.

[26] Emre O Neftci, Hesham Mostafa and Friedemann Zenke. 'Surrogate Gradient Learning in Spiking Neural Networks: Bringing the Power of Gradient-Based Optimization to Spiking Neural Networks'. In: *IEEE Signal Processing Magazine* 36.6 (2019), pp. 51–63. DOI: 10.1109/MSP.2019.2931595.

[27] Michael Pfeiffer and Thomas Pfeil. 'Deep Learning With Spiking Neurons: Opportunities and Challenges'. In: *Frontiers in Neuroscience* Volume 12 - 2018 (2018). ISSN: 1662-453X. DOI: 10.3389/fnins.2018.00774. URL: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2018.00774.

[28] Joseph Redmon et al. 'You only look once: Unified, real-time object detection'. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 779–788.

[29] Hamid Rezatofighi et al. 'Generalized Intersection Over Union: A Metric and a Loss Function'. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, pp. 658–666. DOI: 10.1109/CVPR.2019.00075.

[30] Kaushik Roy, Akhilesh Jaiswal and Priyadarshini Panda. 'Towards spike-based machine intelligence with neuromorphic computing'. In: *Nature* 575.7784 (Nov. 2019), pp. 607–617. ISSN: 1476-4687. DOI: 10.1038/s41586-019-1677-2. URL: https://pubmed.ncbi.nlm.nih.gov/31776490/.

[31] Rahul Sarpeshkar, Lloyd Watts and Carver Mead. 'Refractory neuron circuits'. In: (1992).

[32] Abhronil Sengupta et al. 'Going Deeper in Spiking Neural Networks: VGG and Residual Architectures'. In: *Frontiers in Neuroscience* Volume 13 - 2019 (2019). ISSN: 1662-453X. DOI: 10.3389/fnins.2019.00095. URL: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2019.00095.

[33] Nicolas Skatchkovsky, Hyeryung Jang and Osvaldo Simeone. 'Bayesian Continual Learning via Spiking Neural Networks'. In: *Frontiers in Computational Neuroscience* 16 (2022), p. 1037976. DOI: 10.3389/fncom.2022.1037976.

[34] Qiaoyi Su et al. 'Deep Directly-Trained Spiking Neural Networks for Object Detection'. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. Oct. 2023, pp. 6555–6565.

[35] Zhe Su et al. 'Core Interface Optimization for Multi-core Neuromorphic Processors'. In: *2023 28th IEEE International Symposium on Asynchronous Circuits and Systems (ASYNC)*. 2023, pp. 89–98. DOI: 10.1109/ASYNC58294.2023.10239574.

[36] Simon Thorpe, Arnaud Delorme and Rufin Van Rullen. 'Spike-based strategies for rapid processing'. In: *Neural Networks* 14.6 (2001), pp. 715–725. ISSN: 0893-6080. DOI: https://doi.org/10.1016/S0893-6080(01)00083-1. URL: https://www.sciencedirect.com/science/article/pii/S0893608001000831.

[37] Simon Thorpe, Denis Fize and Catherine Marlot. 'Speed of processing in the human visual system'. In: *Nature* 381.6582 (1996), pp. 520–522. DOI: 10.1038/381520a0.

[38] Pierre de Tournemire et al. 'A Large Scale Event-based Detection Dataset for Automotive'. In: *CoRR* abs/2001.08499 (2020). arXiv: 2001.08499. URL: https://arxiv.org/abs/2001.08499.

[39] Khanin Udomchoksakul, Orathai Sangpetch and Akkarit Sangpetch. 'GPU Performance Tuning and Power Efficiency on the DGX A100 Cluster'. In: *2022 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. 2022, pp. 170–177. DOI: 10.1109/CloudCom55334.2022.00033.

[40] Aayush Atul Verma et al. 'eTraM: Event-based Traffic Monitoring Dataset'. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2024, pp. 22637–22646.

[41] Yulong Yan et al. 'Backpropagation With Sparsity Regularization for Spiking Neural Network Learning'. In: *Frontiers in Neuroscience* 16 (2022), p. 760298. DOI: 10.3389/fnins.2022.760298.

[42] Man Yao et al. 'Scaling Spike-Driven Transformer With Efficient Spike Firing Approximation Training'. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 47.4 (2025), pp. 2973–2990. DOI: 10.1109/TPAMI.2025.3530246.

# Appendix A

# Implementation Specifications

## A.1 DetectorNX-G3-SNN Hyperparameters

| Category | Hyperparameter | Value |
|---|---|---|
| Architecture | Temporal bins ($T$) | 10 |
| | Input channels ($C$) | 2 (polarity) |
| | Input resolution | $180 \times 320$ |
| | Backbone blocks | 4 (SEWResBlock) |
| | Channel progression | $64 \to 128 \to 256 \to 512 \to 512$ |
| | Stride pattern | $2, 2, 2, 1, 1$ (stem, B1, B2, B3, B4) |
| | SE reduction ratio | 16 |
| | Head projection channels | 128 |
| | Adaptive pool size | $8 \times 8$ |
| | Pool strategy | Avg + Max (element-wise) |
| | Detection slots | 5 ($\times$ 4 coords + 1 conf) |
| Neuron Dynamics | Neuron model | LIF (Leaky Integrate-and-Fire) |
| | Membrane time constant ($\tau$) | 4.0 |
| | Firing threshold ($V_{th}$) | 0.3 |
| | Surrogate gradient | ATan |
| | Step mode | Multi-step (`step_mode='m'`) |
| | Temporal aggregation | Sum over $T$ (pre-head) |
| Training | Optimiser | Adam |
| | Learning rate | $2 \times 10^{-4}$ |
| | Weight decay | $1 \times 10^{-4}$ |
| | Epochs | 150 |
| | Batch size | 8 |
| | Precision | AMP (mixed FP16/FP32) |
| | Gradient clipping (max norm) | 1.0 |
| Loss | Coordinate loss | SmoothL1 |
| | Box loss | GIoU ($\lambda_{\text{GIoU}} = 2.0$) |
| | Confidence loss | BCEWithLogitsLoss |
| | Matching strategy | Greedy (max IoU per GT) |
| Regularisation | Firing rate target | 20% |
| | FRR penalty ($\lambda_{\text{FRR}}$) | 5.0 |

## A.2 DetectorNX-G3-CNN Hyperparameters

| Category | Hyperparameter | Value |
|---|---|---|
| Architecture | Temporal bins ($T$) | 10 |
| | Input channels ($C$) | 2 (polarity) |
| | Effective input channels | $T \times C = 20$ (flattened) |
| | Input resolution | $180 \times 320$ |
| | Backbone blocks | 4 (ResBlockCNN) |
| | Channel progression | $64 \to 128 \to 256 \to 512 \to 512$ |
| | Stride pattern | $2, 2, 2, 1, 1$ (stem, B1, B2, B3, B4) |
| | SE reduction ratio | 16 |
| | Head projection channels | 128 |
| | Adaptive pool size | $8 \times 8$ |
| | Pool strategy | Avg + Max (element-wise) |
| | Activation function | ReLU |
| | Detection slots | 5 ($\times$ 4 coords + 1 conf) |
| | Temporal handling | Flattened into channel dim (single pass) |
| Training | Optimiser | Adam |
| | Learning rate | $2 \times 10^{-4}$ |
| | Weight decay | $1 \times 10^{-4}$ |
| | Epochs | 150 |
| | Batch size | 8 |
| | Precision | AMP (mixed FP16/FP32) |
| | Gradient clipping (max norm) | 1.0 |
| Loss | Coordinate loss | SmoothL1 |
| | Box loss | GIoU ($\lambda_{\text{GIoU}} = 2.0$) |
| | Confidence loss | BCEWithLogitsLoss |
| | Matching strategy | Greedy (max IoU per GT) |
| Regularisation | Firing rate regularisation | N/A (no spiking neurons) |
| | Head dropout | 0.0 |
| LR Schedule | Scheduler | ReduceLROnPlateau |
| | Mode / metric | max (EMA IoU) |
| | Reduction factor | 0.5 |
| | Patience | 15 epochs |
| | EMA smoothing ($\beta$) | 0.9 |
| | IoU gate (scheduler activation) | 0.10 |

Table A.2: `DetectorNX-G3-CNN` hyperparameters (11.73M parameters).